Planning at the Speed of Control: Fusing a JEPA World-Model Planner into One GPU Kernel

An equal-hardware optimisation ladder for LeWorldModel on H100, from exact caching and iCEM to a ThunderKittens megakernel

Antoine Nguyen

Student number 24433599

Research Paper

[Affiliation]

Supervisor: [Supervisor name]

October 2026

Abstract

Joint-embedding predictive world models plan by searching over action sequences in a learned latent space, and their planning latency has been the obstacle to using them for control. LeWorldModel (LeWM), the smallest model of the line, made planning fast enough to discuss for control by shrinking the representation to a single token, yet its released planner still takes 622 ms per plan for one robot on an H100. This paper asks how much of that latency can be removed without changing the model, the planner's semantics or its task success, and which kinds of change account for it. We build a seven-rung ladder on one H100, adding one change per rung to the released planner and measuring each rung's end-to-end latency for a single robot and its closed-loop success on the same 300 paired Push-T episodes, with exact McNemar tests and bootstrap intervals. Encoding the observation and goal once per plan and keeping the optimiser on the GPU is exact and removes 40% of the latency (377 ms). The improved cross-entropy method (iCEM), which evaluates 3.19 times fewer candidate rollouts, is 8% slower than CEM in eager PyTorch, where every cost call is launch-bound and costs about 20 ms at 60 candidates as at 300, and 1.38 times faster inside a megakernel; its value depends on the execution model beneath it. We then replace the PyTorch execution with LeWM-MK, one persistent ThunderKittens kernel per plan that runs the ViT encoder, the AdaLN predictor rollout with an exact KV cache, the cost and all 30 iCEM iterations with in-kernel sampling and elite selection, using thread-block-cluster tensor parallelism over distributed shared memory and a static weight stream. A plan takes 46.5 ms (p90 47.0 ms), 13.4 times faster than the reference and 3.9 times faster than the best PyTorch implementation with compilation, TF32 and BF16 autocast (182 ms). The kernel's BF16-operand, FP32-accumulate numerics match the accepted autocast path in cost error, and task success is unchanged at every rung: the final rung differs from the reference by −0.3 points (95% CI −4.3 to +3.7) on 300 paired episodes. A JEPA planner at batch 1 is bound neither by arithmetic nor by memory bandwidth but by the latency of a chain of 900 dependent layer evaluations, and fusing the whole optimiser loop into one launch is the lever that moves it.

Acknowledgements

We thank the authors of LeWorldModel and stable-worldmodel for releasing the model, its planner and its evaluation harness, which made a same-hardware comparison against the reference implementation possible, and the Hazy Research group for ThunderKittens and for the public account of their Llama megakernels, which this work follows in spirit. The GPU time was provided by Modal on rented H100 instances.

Declaration

I declare that this paper is my own work, that it has not been submitted for any other degree or diploma, and that all sources of information and assistance have been acknowledged. Generative AI tools were used as a coding assistant, as a literature search aid and as a writing aid; every source cited was checked against the primary document, and every result reported was produced by the code and evaluation records described in Appendix E. No ethics approval was required: the research involved no human or animal participants.

Table of Contents

Abstract 2
Acknowledgements 3
Declaration 3
1. Introduction 5
1.1 Background 5
1.2 Research significance 6
1.3 Research questions 6
1.4 Literature review 7
1.5 Research gap and contributions 10
2. Methodology 11
2.1 Research design 11
2.2 System under test 11
2.3 Task and episodes 13
2.4 The ladder 13
2.5 Numerical acceptance 17
2.6 Evaluation 17
2.7 Experimental programme 18
2.8 Implementation and infrastructure 19
3. Results 20
3.1 The ladder end to end 20
3.2 Caching and synchronisation: 1.65×, exact 21
3.3 iCEM: no gain in eager PyTorch, 1.38× in the megakernel 21
3.4 The megakernel: 6.9× over eager iCEM, 3.1× over LeWM-Flash 22
3.5 The fused encoder: 1.28×, one launch per plan 23
3.6 Task success is unchanged at every rung 23
3.7 Numerical agreement with the reference 25
3.8 How the kernel reached 41 ms, and what did not work 25
4. Discussion 28
4.1 Answers to the research questions 28
4.2 Why a JEPA planner is a latency problem, not a throughput problem 28
4.3 What the megakernel design needed 29
4.4 Position relative to related methods 30
4.5 Limitations and threats to validity 31
5. Future work 32
6. Conclusion 33
References 34
Appendix A. Per-rung results 40
Appendix B. Configuration 41
Appendix C. Repeatability and the batched regime 43
Appendix D. Compute, cost and incident log 44
Appendix E. Code and data availability 45

1. Introduction

1.1 Background

World models that plan in a learned latent space have become one of the main routes to goal-conditioned robot control without rewards. The joint-embedding predictive architecture (JEPA) is the most compact form of the idea. Instead of reconstructing pixels, a JEPA learns an encoder and a predictor such that the predicted embedding of a future observation matches the encoder's embedding of it (Assran et al., 2023; Bardes et al., 2024). Conditioned on actions, the predictor becomes a dynamics model, and a goal image becomes a target in embedding space: planning is the search for the action sequence whose predicted final embedding lies closest to the goal's. DINO-WM showed that such a model, built on frozen DINOv2 patch features, plans zero-shot on manipulation and navigation tasks (Zhou et al., 2025; Oquab et al., 2024); PLDM made the case for planning with latent dynamics learned from reward-free data (Sobal et al., 2025); and V-JEPA 2-AC carried the approach to real Franka arms from video pre-training (Assran et al., 2025). LeWorldModel (LeWM) is the most recent and the smallest of the line: a 15-million-parameter JEPA trained end to end from pixels with a single Gaussian regulariser, SIGReg (Balestriero & LeCun, 2025), that reaches 96% success on Push-T with a cross-entropy-method planner (Maes, Le Lidec, Scieur, et al., 2026).

Nearly all of these systems plan the same way. A sampling-based optimiser, the cross-entropy method (CEM; Rubinstein, 1999) or a relative, draws hundreds of candidate action sequences, rolls each through the predictor, keeps the best, refits its sampling distribution to them and repeats, and the robot executes the first actions of the result before planning again (Chua et al., 2018; Hafner et al., 2019). The cost of a plan is therefore the cost of a model evaluation multiplied by the number of candidates and iterations, and for the large world models of the line it has been the obstacle to deployment. DINO-WM takes 53 seconds for one CEM call on an A6000 (Zhou et al., 2025); V-JEPA 2-AC takes 16 seconds per action on an RTX 4090 (Assran et al., 2025); a diffusion-based navigation world model takes 30 seconds to simulate one trajectory (Bar et al., 2025). LeWM was designed against exactly this cost. By encoding each image to a single 192-dimensional token rather than hundreds of patch features, it plans up to 48 times faster than foundation-model world models and completes a plan "in under one second" (Maes, Le Lidec, Scieur, et al., 2026).

Under one second is fast for a world model and slow for a controller. A planner that replans every few control steps must return within the period it plans for; a manipulator closing a contact-rich loop at ten hertz has a budget of 100 milliseconds for sensing, planning and actuation together. On an H100, the released LeWM planner takes 622 ms to produce one plan for one robot. The reasons are not what they are for the larger world models. LeWM's model is so small that a cost evaluation of 300 candidates is a fraction of a millisecond of tensor-core arithmetic, and its weights fit in the GPU's cache. Its plan is slow because the planner, as released, recomputes work it could reuse, issues every operator of every rollout from Python, returns to the CPU in every iteration, and walks a chain of 900 dependent transformer-layer evaluations per plan through a framework built for large batches. The latency of a LeWM plan is a systems problem, and the parts of it that are not are the planner's choice of how many candidates to evaluate.

This paper is about removing that latency without changing what the planner computes. We hold the model, its weights and the semantics of its planner fixed, and we change the

inference stack one measured step at a time on one H100: first the exact changes that remove redundant work, then the improved cross-entropy method (iCEM; Pinneri et al., 2021), which evaluates three times fewer candidates, and finally a single persistent GPU kernel, a megakernel, that runs the whole planner from the raw camera frame to the action distribution in one launch. At every step we measure what a deployed controller would see, the end-to-end latency of one replanning call for one robot, and what the task would see, closed-loop success on the same 300 Push-T episodes.

1.2 Research significance

The significance of the work is threefold. Scientifically, the planning latency of JEPA world models has been reported only as a by-product of model design: each paper that made planning faster did so by changing the model or the planner (Section 1.4), and none held both fixed and asked how much of the remaining latency is the execution of the planner rather than its mathematics. A measured answer for the smallest model of the line tells the field where the floor is for this class of planner, and which of the familiar levers (fewer samples, lower precision, compilation, fusion) actually move it.

Methodologically, the paper separates three kinds of change that are usually reported together, exact, planner-level and numeric, and measures each on the same hardware against the same reference, with paired closed-loop evaluation for every rung. That separation is what reveals that a planner-level change can be worth nothing in one execution model and a third of its nominal value in another, which a ladder measured in a single execution model would miss.

Practically, a 46.5-millisecond plan for one robot on one GPU, with unchanged task success, moves JEPA planning from the time scale of deliberation to the time scale of control. It does so for the released model and planner, with no retraining and no change to the planner's hyperparameters, so the gain is available to any system that uses LeWM as released, and the design principles behind it, a static weight stream, cluster-level tensor parallelism and an optimiser loop that never leaves the GPU, apply to sampling-based planning with any small world model.

1.3 Research questions

The primary question is how far the batch-1 planning latency of a JEPA world model can be reduced on a datacentre GPU without changing the model, the planner's semantics or its task success, and which kinds of change account for the reduction. We decompose it into four questions.

- **RQ1.** How much of the reference planner's latency is redundant work that an exact change removes?
- **RQ2.** Does the improved cross-entropy method make planning faster, by how much, and does it change task success?
- **RQ3.** How much does executing the whole planner as one persistent kernel gain over the best PyTorch implementation of the same planner?
- **RQ4.** Do the numeric changes of the megakernel and of a fused encoder preserve the planner's accuracy, measured against FP32 on identical inputs and in closed loop on paired episodes?

The hypotheses at the start of the H100 programme were that caching would be exact and worth a large share of the reference latency, that iCEM's gain would scale with its 3.19-fold reduction in model evaluations, that a megakernel would beat the compiled BF16 PyTorch path by at least a factor of two, and that none of the steps would change task success. Sections 3 and 4 report that the first, third and fourth held, and that the second was false in eager PyTorch and held at less than half its nominal size in the megakernel.

1.4 Literature review

1.4.1 JEPA world models and planning in latent space

The joint-embedding predictive architecture was introduced for self-supervised image representation learning: I-JEPA predicts the representations of masked target blocks from a context block rather than reconstructing their pixels (Assran et al., 2023), and V-JEPA extends feature prediction to video (Bardes et al., 2024). The step from representation learning to control was taken by conditioning the predictor on actions. DINO-WM trains a predictor over frozen DINOv2 patch features and plans with CEM-based model-predictive control by minimising latent distance to a goal, reaching 90% success on Push-T (Zhou et al., 2025; Oquab et al., 2024). PLDM plans with a JEPA-style latent dynamics model trained on reward-free offline data and generalises better than model-free goal-conditioned reinforcement learning (Sobal et al., 2025). V-JEPA 2 pre-trains on over a million hours of video, post-trains an action-conditioned predictor on 62 hours of robot data, and plans image-goal manipulation on real arms with CEM at 800 samples and 10 iterations (Assran et al., 2025). A systematic study of what drives planning success in JEPA world models found the CEM family the best planner on Push-T and gradient-based planners poor there, and reported high sensitivity to the planner's population settings, but no planning times (Terver et al., 2026).

LeWM is the model we study (Maes, Le Lidec, Scieur, et al., 2026). It trains a ViT-tiny encoder (Dosovitskiy et al., 2021) and a 6-layer transformer predictor with AdaLN action conditioning (Peebles & Xie, 2023) end to end from pixels, with a prediction loss and the SIGReg regulariser, which pushes embeddings towards an isotropic Gaussian and replaces the stop-gradient and teacher-student heuristics of earlier JEPAs (Balestriero & LeCun, 2025). It plans with CEM at 300 samples, 30 elites and 30 iterations on Push-T, horizon 5 with a frame-skip of 5, and its evaluation and reference planners are those of the stable-worldmodel platform (Maes, Le Lidec, Facury, et al., 2026). Its descendants make planning cheaper by changing the representation or the model: Causal-JEPA uses object-level latents with about 1% of the input features of patch-based models (Nam et al., 2026), temporal straightening regularises latent trajectories so that gradient-based planning becomes stable (Y. Wang et al., 2026), and representation geometry has been shown to matter for planning with both CEM and iCEM (Hu et al., 2026). The JEPA planning literature thus treats latency as a property of the model. Where it is reported it is coarse and secondary: 53 s per DINO-WM CEM call, 16 s per V-JEPA 2-AC action, about 1.2 s per LeWM plan against about 92 s per DINO-WM plan in an independent measurement (Hu et al., 2026), and "under one second" for LeWM in its own paper.

1.4.2 Sampling-based trajectory optimisation: from CEM to iCEM

The cross-entropy method was introduced for rare-event estimation and combinatorial optimisation and adapted to continuous optimisation by sampling from a parametric

distribution, selecting elites and refitting the distribution to them (Rubinstein, 1999; de Boer et al., 2005). PETS established CEM over learned probabilistic dynamics as a standard for model-based reinforcement learning (Chua et al., 2018), and PlaNet made CEM planning in a learned latent space from pixels the template for latent model-predictive control (Hafner et al., 2019). TD-MPC and TD-MPC2 plan in the latent space of a decoder-free world model with a terminal value function and the related MPPI optimiser (Hansen et al., 2022, 2024); model-predictive path integral control was itself designed from the outset as a GPU-parallel sampling method (G. Williams, Aldrich, & Theodorou, 2017) and run in real time through neural-network dynamics (G. Williams, Wagener, et al., 2017).

The improved cross-entropy method, iCEM, is the sample-efficient variant we adopt (Pinneri et al., 2021). It adds four features to CEM: temporally correlated, coloured-noise sampling; keeping a fraction of the elites from one iteration to the next and shifting them in time across replans; executing the best action rather than the mean; and a population that decays exponentially across iterations, $N_i = \max(N\,\gamma^{-i}, 2K)$. Together they reduced the samples needed by 2.7 to 22 times on its benchmarks and made real-time planning possible with PlaNet's models in PyTorch. iCEM has been run on LeWM as one of several solvers in an evaluation of representation geometry, without a decay schedule or elite retention and not to reduce latency (Hu et al., 2026), and it is among the solvers stable-worldmodel provides. The alternatives that cut samples by adding gradients, CEM interleaved with gradient steps on the action sequences (Bharadhwaj et al., 2020) and CEM-GD (Huang et al., 2021), illustrate a point central to this paper: CEM-GD uses about a hundred times fewer samples per step than a 5,000-sample CEM and is only 25% faster than it, and it remains several times slower than CEM with 50 to 500 samples, because the per-iteration cost of a planner does not scale with its sample count once the sample count is small. Gradient-based planning with world models has been studied directly and found competitive in sample-efficient settings (S V et al., 2023), but CEM remains the best planner on Push-T for LeWM and for JEPA world models generally (Maes, Le Lidec, Scieur, et al., 2026; Terver et al., 2026).

1.4.3 Making world-model planning cheaper

Three routes to cheaper planning have been taken, and each changes something the ladder holds fixed. The first changes the model. Fast LeWorldModel retrains LeWM to predict all action prefixes of a candidate in one parallel pass instead of five autoregressive steps, which halves CEM time on an RTX 4090 (33.7 to 16.1 s over its evaluation workload) at the cost of retraining (Gao & Xu, 2026). Distillation of a diffusion navigation world model cuts its trajectory simulation from 30 to 0.4 s with heavy approximation (Bar et al., 2025). The second changes the planner to use gradients (Bharadhwaj et al., 2020; Huang et al., 2021; S V et al., 2023; Y. Wang et al., 2026), which on Push-T has so far cost success. The third amortises planning into a policy: POPLIN initialises or replaces planning with a policy network (T. Wang & Ba, 2020), and a goal-conditioned inverse-dynamics model trained over a pretrained LeWM matches or beats CEM in 7 of 8 settings at 100 to 130 times lower cost per decision (Nguyen et al., 2026). Each route is complementary to ours, and each gives up something, the released weights, the planner's success or the explicit search, that we keep.

In robotics outside world models, GPU-parallel sampling MPC is mature and fast. STORM computes MPPI-style joint-space control for a Franka arm in under 8 ms (Bhardwaj et al., 2022), cuRobo solves collision-free motion generation in about 50 ms through custom GPU kernels (Sundaralingam et al., 2023), and MPPI-Generic offers a combined CUDA kernel

that fuses the dynamics and cost of a rollout with intermediates in shared memory (Vlahov et al., 2024). These systems reach millisecond planning with analytic or small hand-written dynamics, and the lesson they teach, that planning reaches the time scale of control only through custom kernels, has not been applied to a transformer world model with an image encoder.

1.4.4 Small-batch inference: launch overhead, data movement and fusion

The roofline model bounds a kernel's throughput by its arithmetic intensity against the machine's memory bandwidth (S. Williams et al., 2009), and the analysis of transformer inference at small batch has shown it to be dominated by memory traffic and latency rather than arithmetic (Pope et al., 2023; Ivanov et al., 2021). A JEPA planner at batch 1 sits below both roofs: its kernels are too small to saturate either the tensor cores or the memory system, and its time goes to launching them and to waiting between them. CUDA Graphs reduce the CPU cost of a launch, from 9.6 µs per kernel with a synchronisation after each launch to 3.4 µs on a V100 in NVIDIA's own measurement, but they do not remove the gaps between kernels (Gray, 2019; NVIDIA, 2026). PyTorch 2's compiler captures graphs and generates fused Triton kernels (Ansel et al., 2024; Tillet et al., 2019), and it is the stack of the strongest PyTorch rung of our ladder; its unit of fusion remains a region of operators inside one model call, with the optimiser loop left on the host.

Fusion inside an operator is the other half of the lineage. FlashAttention computes exact attention in tiles held in on-chip memory without materialising the score matrix (Dao et al., 2022), FlashAttention-2 improved its work partitioning across warps (Dao, 2024), and FlashAttention-3 used Hopper's asynchronous tensor-core instructions (WGMMA), its Tensor Memory Accelerator and warp specialisation to reach 85% utilisation in BF16 (Shah et al., 2024). Reduced precision with FP32 accumulation is the standard numerics of this work (Micikevicius et al., 2018), and BF16 keeps FP32's dynamic range without changes to hyperparameters (Kalamkar et al., 2019); our megakernel adopts the same class.

1.4.5 Persistent kernels and megakernels

A persistent kernel launches only as many thread blocks as the GPU can hold and loops over work inside them (Gupta et al., 2012). Persistent RNNs used the idea to keep a recurrent network's weights on chip across timesteps at small batch (Diamos et al., 2016), the closest neural-network precedent for a planner whose small predictor is applied 150 times per plan. The idea returned for language-model decoding in 2025 as the megakernel: a whole forward pass in one launch. Hazy Research fused the batch-1 decode of Llama-3.2-1B into one ThunderKittens kernel with an on-GPU instruction interpreter and paged shared memory, removing the "bubbles" between kernels and reaching forward passes under a millisecond on an H100 (Spector, Juravsky, Sul, Dugan, et al., 2025), and then extended it to a tensor-parallel Llama-70B on eight GPUs (Spector, Juravsky, Sul, Lim, et al., 2025). FlashFormer fuses the whole transformer forward pass for low-batch inference (Nrusimha et al., 2025). On the compiler side, Mirage searches for fused kernels automatically (Wu et al., 2025), MPK compiles model inference into one persistent megakernel with in-kernel task scheduling and reports up to 1.7 times lower latency (Cheng et al., 2026), and Event Tensor addresses the shape- and data-dependent dynamism that megakernels must support (Jin et al., 2026).

ThunderKittens is the framework in which our kernel is written: it exposes Hopper's WGMMA and bulk asynchronous copies through warp- and warpgroup-level tiles, and its kernels

match cuBLAS on GEMM and FlashAttention-3 on attention (Spector, Arora, et al., 2025). The Hopper features the kernel depends on are documented by NVIDIA, thread-block clusters of up to 16 blocks with distributed shared memory, asynchronous transaction barriers and bulk copies on a 132-SM H100 (Andersch et al., 2022; NVIDIA, 2022), and characterised independently by micro-benchmarks (Luo et al., 2024).

1.5 Research gap and contributions

Against this literature the gap is precise. Planning with JEPA world models is CEM-family sampling at its core, and its latency has been reduced only by changing the model, the planner or both; no work has held the model and the planner's semantics fixed and measured what the execution of the planner costs. iCEM has been applied to LeWM as an evaluated solver, but its value for latency, which depends on the execution model beneath it, has not been measured. Megakernels have been built for language-model decoding, where the unit of fusion is one forward pass, and GPU sampling MPC has fused single rollouts with analytic dynamics; no work fuses an entire sampling-based planning loop, with a neural encoder, a multi-step transformer rollout, cost, elite selection and distribution updates across every iteration, into one kernel. The paper makes five contributions.

1. A measured optimisation ladder for JEPA world-model planning on one H100, in which every rung adds one change to the reference LeWM planner and is evaluated for batch-1 end-to-end latency and for paired closed-loop success, separating exact, planner-level and numeric changes.
2. The finding that iCEM's latency value depends on the execution model: it is 8% slower than CEM in eager PyTorch, where cost calls are launch-bound, and 1.38 times faster inside a megakernel, with task success unchanged in both.
3. LeWM-MK, to our knowledge the first megakernel for world-model planning: one persistent ThunderKittens kernel that runs the ViT encoder, the action encoder, the AdaLN predictor rollout with an exact KV cache, the cost and all 30 iCEM iterations with in-kernel sampling and elite selection, using cluster-level tensor parallelism over distributed shared memory and a static weight stream with no producer warp.
4. A batch-1 plan of 46.5 ms against 622 ms for the reference (13.4 times) and 182 ms for the best PyTorch implementation (3.9 times), with p90 within 0.6 ms of the median, and no detectable change in task success on 300 paired episodes (−0.3 points, 95% CI −4.3 to +3.7).
5. A reproducible protocol with committed per-call latency records, per-episode outcomes, numerical parity records and analysis scripts, and an account of the budget it consumed.

2. Methodology

2.1 Research design

The research is quantitative and experimental. We take one released JEPA world model and its released planner, change the inference stack one step at a time, and measure each step with two instruments: the end-to-end latency of a replanning control call for a single robot on one GPU, and closed-loop task success on a fixed, paired set of Push-T episodes. Four design principles were fixed before the H100 programme spent any GPU time, and they are what separate a measured ladder from a collection of speed-ups.

First, every rung adds exactly one change to the rung below it, and every rung runs on the same hardware in the same container image. The reference is the stock LeWM planner as released, not a straw man and not a result transferred from another GPU. The steps of the ladder are then attributable: a rung's gain is the ratio of two measurements that differ in one change.

Second, the primary metric is batch-1 end-to-end plan latency. A deployed controller serves one robot, and it waits for the whole call, not for the planner kernel inside it. We therefore time the replanning `policy.get_action()` call from observation in to action out, including image preprocessing, encoding, the solve and the copy back to the host, and we do it with one environment per process so that no batching amortises any cost. Kernel-level timings are reported only to explain the end-to-end numbers.

Third, accuracy is measured in the loop, and comparisons are paired. A change that alters the numerics (BF16 operands) or the optimiser (iCEM) could change what the robot does even when every cost it computes is close to the reference. Every rung is evaluated on the same 300 Push-T episodes, whose start states are a deterministic function of the seed, so that two rungs differ only in the planner that acts on identical initial conditions. Evaluation episodes run 100 environments at a time, which is an evaluation convenience only: each environment's planner mathematics is the same whether one or a hundred are solved together.

Fourth, the change classes are kept separate. We distinguish exact changes, which compute the same function in a different order (caching, removing host synchronisation); planner changes, which change the optimisation algorithm (iCEM); and numeric changes, which compute the same function at lower precision or in a different arithmetic order (BF16 operands, fused kernels). Each class has its own acceptance test: exact changes must reproduce the reference costs and rankings on fixed candidates, numeric changes must stay within the cost and ranking error of the accepted BF16 autocast path, and every change must leave paired closed-loop success within noise of the reference.

2.2 System under test

We study LeWorldModel (LeWM; Maes, Le Lidec, Scieur, et al., 2026), a joint-embedding predictive world model trained end to end from pixels, in its released Push-T configuration, planned by the cross-entropy method of stable-worldmodel (Maes, Le Lidec, Facury, et al., 2026) exactly as the authors release it. The facts of the model and planner that the optimisations depend on are listed in Table 1; each was read from the released code and checkpoint rather than from the paper.

Table 1. The system under test as released. Facts from the LeWM and stable-worldmodel repositories and the Push-T evaluation configuration.

item	value
encoder	ViT-tiny: 12 layers, width 192, 3 heads, patch 14 on 224 × 224 images (256 patches + CLS = 257 tokens), LayerNorm $\varepsilon = 10^{-12}$; CLS output through a BatchNorm projector to a 192-dimensional embedding
predictor	6-layer transformer, width 192, 16 heads × 64, MLP 2,048, AdaLN-zero conditioning on the action embedding (Peebles & Xie, 2023), history window of 3 latent frames, causal attention; output through a BatchNorm projector (pred_proj)
action encoder	one-dimensional convolution with kernel 1 and a two-layer MLP: 10 → 768 → 192 per action block
action space	2-dimensional end-effector displacement, grouped into blocks of 5 (frame-skip 5), so each plan step is a 10-dimensional action block
planning objective	squared Euclidean distance between the predicted embedding after the horizon and the goal embedding
planner (reference)	CEM: 300 candidate sequences, 30 iterations, top-30 elites, horizon 5 blocks, receding horizon 5, Gaussian initialisation with variance scale 1
control loop	50 policy calls per episode; a plan is computed on call 1 and on call 26 and its 5 action blocks are executed over the following calls; the other 48 calls pop buffered actions
evaluation	Push-T from the expert dataset: start state from the dataset, goal 25 steps later, budget 50 policy calls, success by the environment's coverage predicate
precision as released	FP32 parameters and arithmetic, eager PyTorch

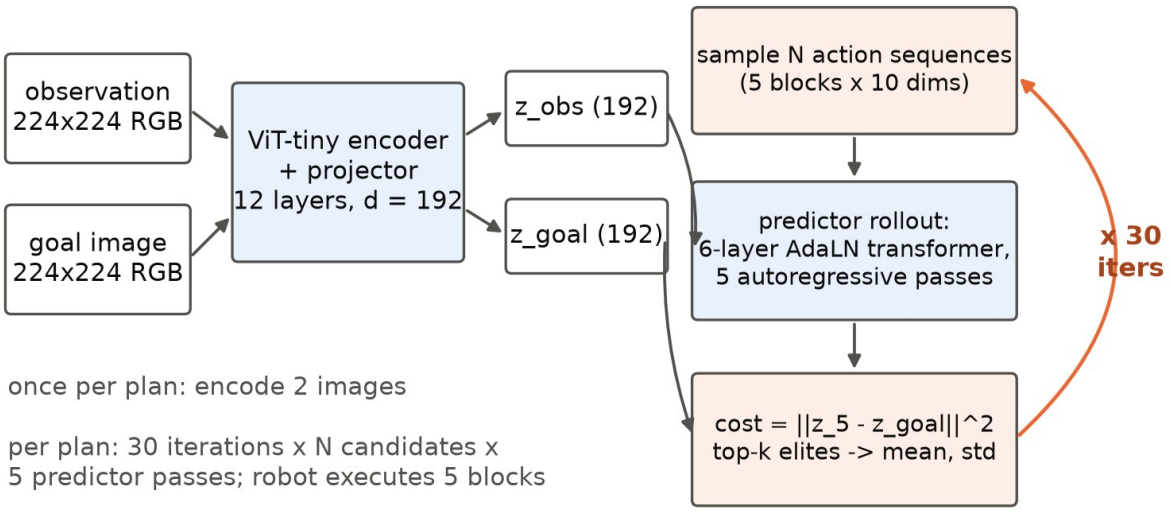


Figure 1. LeWM planning on Push-T. The encoder maps the current observation and the goal image to 192-dimensional embeddings; the planner samples candidate action sequences, rolls each through the predictor for five autoregressive steps, scores the final embedding against the goal and refits the sampling distribution to the best candidates. The reference planner runs 30 iterations of 300 candidates per plan.

Two properties of this system define the optimisation problem. The model is small: the whole model holds about 15 million parameters, about 5 million of them in the encoder (Maes, Le Lidec, Scieur, et al., 2026), so one cost evaluation of 300 candidates is under 100 GFLOP, less than a tenth of a millisecond of H100 tensor-core time, and all the weights fit in the

GPU's 50 MB L2 cache. And the planner is deep: one plan is 30 sequential iterations of a 5-step autoregressive rollout through 6 layers, a chain of 900 dependent transformer-layer evaluations per plan whose individual matrix products are 64 to 300 rows tall. Such a workload is bound neither by arithmetic nor by memory bandwidth on a datacentre GPU; it is bound by latency, by how fast the hardware can walk a long chain of small dependent steps. Every rung of the ladder is an attack on a different term of that latency.

2.3 Task and episodes

Push-T requires the agent to push a T-shaped block into a target pose on a table, and it has become the common manipulation benchmark for planning with learned world models (Florence et al., 2022; Chi et al., 2023; Zhou et al., 2025; Maes, Le Lidec, Scieur, et al., 2026). We use stable-worldmodel's `swm/PushT-v1` environment with episodes drawn from the expert training set as in the LeWM evaluation: the environment is set to a recorded state, the goal is the recorded state 25 steps later, and the agent has 50 policy calls to reach it. Two episode sets are used and both are fixed by seed.

The **quality set** is 300 episodes, 100 for each of seeds 0, 1 and 2, the protocol of the LeWM-Flash study that preceded this one. The reference planner solves 87% of these episodes, below the 96% that LeWM reports for CEM on Push-T; we use the evaluation configuration released with the code (start states from the expert set, goal offset 25, budget 50) and did not investigate the difference, which is common to every rung. Episode start states and goals are a deterministic function of the seed, so every rung sees the same 300 episodes and every comparison between rungs is paired by episode.

The **latency set** is 61 single-environment episodes, seeds 1000 to 1060, run one environment per process. Each episode replans at calls 1 and 26 unless it succeeds before call 26, which gives one or two measured plans per episode. The first seed is dropped as process warmup, leaving 72 to 76 measured plans per rung. Calls that do not replan are recorded separately as step latency.

2.4 The ladder

The ladder has five main rungs and two side rungs (Table 2). R0 is the reference as released. R1 and R2 are the planner-level changes that the LeWM-Flash study accepted on an L40S, R1 exact and R2 changing the optimiser. R2b is LeWM-Flash itself, which adds the kernel- and precision-level changes of that study (`torch.compile`, TF32, BF16 autocast) to R2; it is the strongest PyTorch implementation we have and the baseline the megakernel has to beat. R3 replaces R2b's kernel and precision stack with one persistent megakernel that runs the predictor and the whole iCEM loop. R4 moves the encoder into the same launch. R3-CEM runs the megakernel with the reference CEM schedule, to price iCEM in an execution model where it can pay.

Table 2. The H100 ladder. Each main rung adds one change to the rung above it; kind is the change class of Section 2.1.

rung	change	kind	numerics of the planner
R0	reference LeWM: stock stable-worldmodel CEM 300 × 30, re-encodes observation and goal inside every cost call	—	FP32 eager
R1	encode observation and goal once per plan; keep CEM bookkeeping on the GPU with no host	exact	FP32 eager

rung	change	kind	numerics of the planner
	synchronisation inside the loop		
R2	bucketed iCEM: 300 → 60 candidates, top-30, 5 retained elites (2,820 evaluations per plan instead of 9,000)	planner	FP32 eager
R2b	LeWM-Flash: R2 + torch.compile of the cost function + TF32 matmuls + BF16 autocast of the rollout	numeric	BF16 operands, FP32 accumulation
R3	megakernel: predictor, action encoder, AdaLN, cost, top-k and the elite update for all 30 iterations in one launch	numeric	BF16 operands, FP32 accumulation and residual
R4	R3 + the ViT encoder and projector fused into the same launch on raw uint8 frames; goal embedding cached per episode	numeric	as R3; encoder BF16 operands, FP32 accumulation
R3-CEM	R3's megakernel running the reference CEM schedule (300 × 30, no retained elites)	side rung	as R3

2.4.1 Exact caching and synchronisation (R1)

The released planner computes the cost of a batch of candidates by re-encoding the observation and the goal for every call, 60 image encodings per plan for 2 distinct images, and it reads candidate costs back to the CPU on every iteration to rank them. R1 encodes both images once per plan, passes the embeddings into a tensor-only cost function, and keeps the ranking, elite selection and distribution update on the GPU, so that the only host synchronisation of the plan is the final action. On fixed candidates the change reproduces the reference costs with zero difference and identical rankings (LeWM-Flash exactness check, L40S).

2.4.2 Improved cross-entropy method (R2)

iCEM (Pinneri et al., 2021) adds to CEM a decaying population, elite retention across iterations, and optionally coloured noise, momentum and clipping. The LeWM-Flash study tested each feature on Push-T and accepted three: a population that decays geometrically by a factor of 1.25 per iteration from 300 to a floor of 60 (twice the elite count), rounded up to a multiple of 60 so that only five distinct batch shapes occur; the top 30 candidates as elites; and the 5 best elites of each iteration retained into the next population. Coloured noise, clipping and momentum lowered success or wall time on Push-T and were rejected, as were shorter schedules of 20 and 25 iterations. The resulting schedule (Figure 2) evaluates 2,820 candidate rollouts per plan against CEM's 9,000, 3.19 times fewer, at the same horizon, iteration count and elite count.

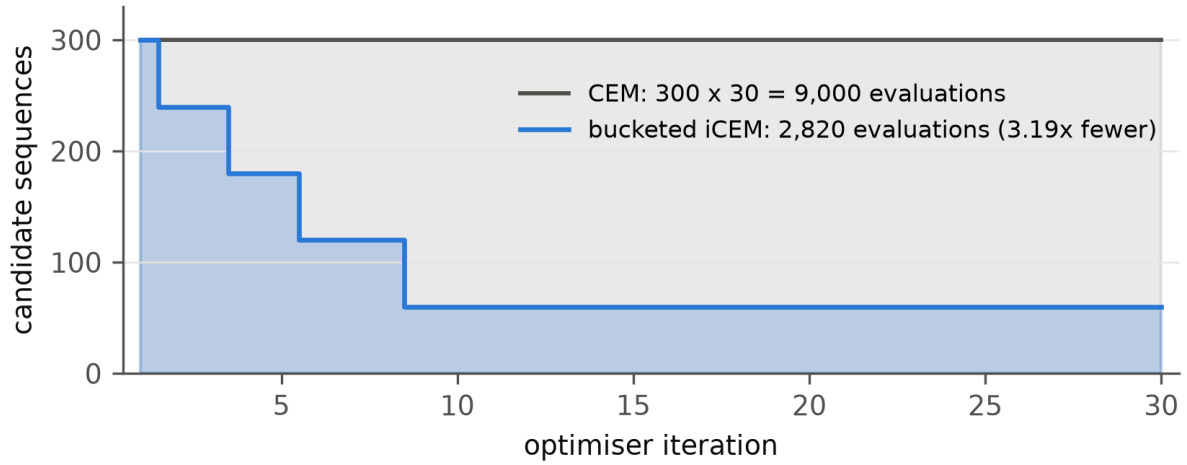


Figure 2. Candidate rollouts per optimiser iteration. CEM evaluates 300 candidates in every one of 30 iterations; bucketed iCEM starts at 300 and decays by 1.25 per iteration to a floor of 60, rounded to multiples of 60.

2.4.3 The megakernel (R3)

R3 keeps every planner-level decision of R2 and discards the PyTorch execution of it. One persistent CUDA kernel, written with ThunderKittens (Spector, Arora, et al., 2025) for the Hopper architecture (Andersch et al., 2022), runs a whole plan: all 30 iterations of sampling, rollout, cost and distribution update, with the host seeing only the final mean and standard deviation. Figure 3 shows its structure; the design follows from the latency-bound character of the workload rather than from FLOPs.

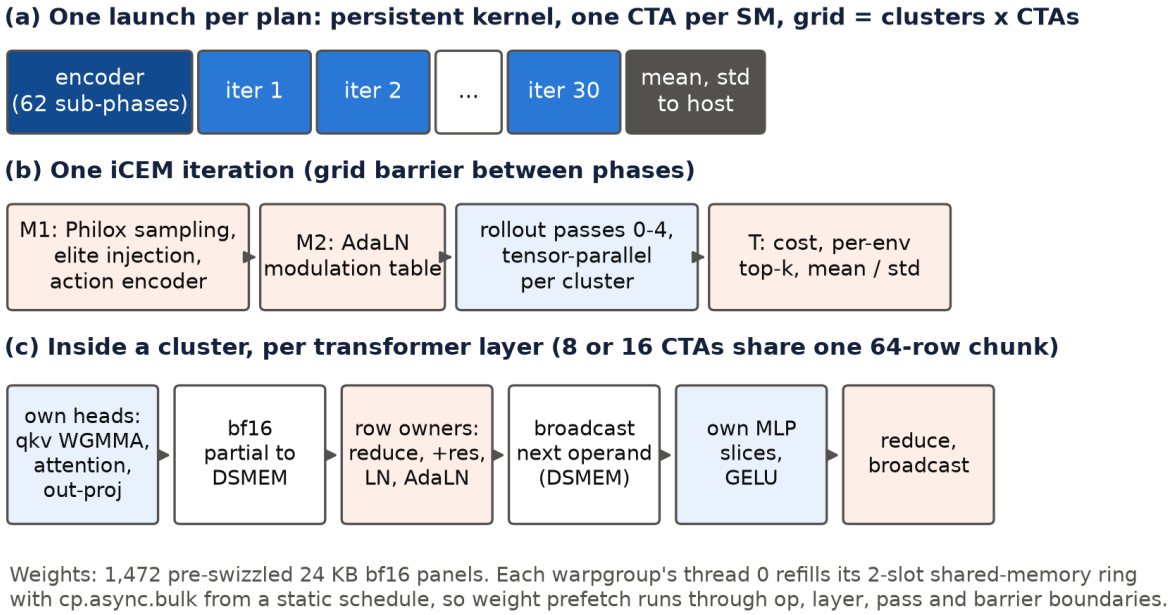


Figure 3. Structure of the megakernel. (a) One launch per plan; R4 adds the encoder phase at the front. (b) The phases of one iCEM iteration, separated by grid barriers. (c) The tensor-parallel schedule of one transformer layer inside a thread-block cluster: each CTA computes its own heads and MLP slices, row-owner CTAs reduce the partial results over distributed shared memory, apply the residual, LayerNorm and AdaLN modulation, and broadcast the next operand.

Persistent grid and phases. The kernel launches one 256-thread CTA (two warpgroups) per streaming multiprocessor, all co-resident, grouped into thread-block clusters of 8 or 16 CTAs (15 or 7 clusters on a 132-SM H100). Each iteration runs four phases separated by grid barriers: M1 draws candidates in the kernel from a counter-based Philox stream (Salmon et al., 2011) as mean plus standard deviation times noise, with slot 0 set to the mean and the next 5 slots to the retained elites, and runs the action encoder; M2 computes the AdaLN modulation table for every candidate, timestep and layer once; the rollout phase runs the five autoregressive predictor passes; and T computes the costs, a per-environment top-k and the elite mean and unbiased standard deviation, which it publishes for the next iteration.

Tensor parallelism inside a cluster. A cluster owns a 64-row chunk of candidate tokens. Each CTA owns 16/CL of the attention heads and 2,048/CL of the MLP and pred_proj hidden units, computes its share of every GEMM with Hopper warpgroup matrix-multiply-accumulate instructions (WGMMA), and writes a BF16 partial into its own shared memory. Row-owner CTAs then reduce the partials across the cluster through distributed shared memory (DSMEM), add the FP32 gated residual, apply LayerNorm and the AdaLN modulation, and broadcast the next GEMM operand into every CTA's shared memory. Each half-layer costs two cluster barriers. We implemented the cluster barrier with mbarriers, one release fence and relaxed remote arrives, which measured 0.65 µs against 3.6 µs for a barrier with a release on every arrive; the grid barrier measured 1.6 µs.

Weights as a static stream. All weights are folded on the host (inner LayerNorm affines, the BatchNorm of pred_proj and the kernel-1 convolution go into adjacent GEMMs) and packed into 964 panels of 24 KB (BF16 tiles of 192 × 64 or 64 × 192), each pre-swizzled in the exact ThunderKittens shared-tile layout so that a single bulk asynchronous copy (cp.async.bulk) lands it ready for WGMMA. Because the sample schedule is fixed, the sequence of panels a CTA needs over the whole plan is a static function of the schedule. Each warpgroup replays that sequence with a resumable iterator and its thread 0 refills a two-slot ring the moment the warpgroup releases a slot, so weight prefetch runs through operation, layer, pass and barrier boundaries without a dedicated producer warp. That choice is forced by the register file: a ninth warp caps every thread at 168 registers, eight warps keep 255.

Exact KV reuse. The predictor's history window grows from one to three frames over passes 0 to 2 and then slides. In the growing passes the keys and values of earlier positions are those of the previous pass, so the kernel caches them and computes 9 token rows per candidate over the five passes instead of 12, with no approximation.

Numerics. The kernel follows the BF16 autocast path of R2b: BF16 GEMM operands with FP32 accumulation, an FP32 residual stream and FP32 LayerNorm statistics, BF16 AdaLN modulation, queries, keys and values, FP32 costs and iCEM statistics. GELU uses the exact error function through the approximation 7.1.26 of Abramowitz and Stegun (1964) (absolute error $\leq 1.5 \times 10^{-7}$) rather than the tanh form. The kernel is therefore mathematically equivalent to the reference and not bit-exact, and its noise comes from its own Philox stream, so its plans differ from the PyTorch solver's draw for draw while following the same distribution.

2.4.4 The fused encoder (R4)

In R3 the encoder still runs in PyTorch at FP32, preceded by the CPU image transform of the policy. R4 adds a 62-sub-phase encoder at the front of the same launch: the kernel reads the raw uint8 frame, normalises it, computes the patch embedding as a GEMM, runs the 12 ViT layers with a FlashAttention-style online softmax (Dao et al., 2022) over the 257 tokens, and applies the projector. The goal image of an episode does not change, so its embedding is cached under a hash of its bytes and only the observation is encoded on later plans. The encoder weights join the same panel stream, which grows from 964 to 1,472 panels (36 MB). A plan is then one kernel launch from image to action distribution.

2.5 Numerical acceptance

Before any closed-loop evaluation, every numeric rung was compared against the FP32 eager reference on fixed inputs. For the predictor and planner (R2b, R3, R4) we evaluate the cost of identical candidate sets drawn at the iCEM population sizes and at full and reduced sampling variance, eight (problem, population) cases in total, and record the maximum relative cost error and the overlap of the top-30 sets with the FP32 top-30. For the encoder (R4) we encode 32 Push-T frames and record the relative L2 error and minimum cosine similarity of the embeddings against FP32, and the cost error and top-30 overlap that result from planning with the fused embeddings. The acceptance criterion was set by the accepted BF16 autocast path of LeWM-Flash: a megakernel error at or below that path's on the same cases.

2.6 Evaluation

2.6.1 Latency protocol

Table 3 gives the latency protocol. The timer is a pair of CUDA events around `policy.get_action()`, recorded by the benchmark harness for every call and classified as a plan or a step by whether the solver ran. All rungs share one container image and the same process structure; each latency seed starts a fresh process, builds the world and loads the model, and for R2b compiles and warms the cost function at every bucketed shape before the first timed call.

Table 3. Batch-1 latency protocol.

item	setting
hardware	one NVIDIA H100 80GB HBM3 (SXM, 132 SMs), Modal
software	PyTorch 2.11.0 with CUDA 12.8, the same container image for every rung; ThunderKittens at commit `67845f5f`; kernel built with `nvcc` for `sm_90a`
episodes	61 one-environment Push-T episodes, seeds 1000 to 1060; first seed dropped as warmup
timed quantity	end-to-end replanning `policy.get_action()` call (preprocessing, encoding, solve, action to host), CUDA events
plans per rung	72 to 76 (calls 1 and 26 of each remaining episode, while it runs)
statistics	median (p50) with a bootstrap 95% interval over plans (10,000 resamples), p90, mean, minimum and maximum; step-call median
repeats	two complete passes of the ladder (Phase A without R4, and the final pass with all seven rungs)

2.6.2 Closed-loop protocol

Each rung was evaluated on the 300 episodes of the quality set, 100 environments at a time, with the planner configuration of Table 2 and nothing else changed. The reference and LeWM-Flash quality runs were made on the same H100 configuration before the ladder (Section 2.7) with exactly the ladder's flags. Success is the environment's predicate at the end of the 50-call budget.

2.6.3 Statistics

The primary accuracy quantity is the success rate over the 300 paired episodes with a Wilson 95% interval (Wilson, 1927), about ±4 points at 87% success. Comparisons between rungs are paired by episode: we report the discordant-episode counts, an exact McNemar test (McNemar, 1947) and a paired bootstrap 95% interval on the difference in success rate with 10,000 resamples over episodes (Efron & Tibshirani, 1993). The ladder's claim on accuracy is a claim of no detectable change, so we state its resolution: at the observed discordance of 12 to 13% of episodes, 300 pairs detect a paired difference of about 6 points at 80% power and 5% two-sided significance, and a 2-point difference would need about 2,400 pairs (Connor, 1987). For latency we report the median of the plan distribution and a bootstrap interval on it, because the distributions of the PyTorch rungs are skewed and in one case bimodal. Gains are ratios of medians.

2.7 Experimental programme

The programme ran in three phases, of which the first predates this study.

- **Phase 0, LeWM-Flash on an L40S (June 2026).** A ledger of PyTorch-level changes to the released planner, each run as a paired 3 × 100 Push-T sweep against the reference: embedding caching and synchronisation removal (accepted), rollout preallocation and an action-embedding cache (rejected), compilation of the cost function (accepted), compiled and static whole-loop CEM variants (rejected), TF32 matmuls (accepted), receding-horizon warm starts of the CEM distribution (rejected: success fell to 34–42%), iCEM with its feature ablations (accepted in the form of Section 2.4.2), iCEM with gradient refinement of the elites (rejected: better model cost, no success gain, slower) and BF16 autocast of the rollout (accepted). The accepted stack is R2b. Its rejected paths are the reason R1 and R2 have the form they do.
- **Phase 1, the megakernel (16 October 2026).** The kernel was developed on H100 against the FP32 reference and the BF16 path, with the tuning steps of Section 3.6, and evaluated on the 300-episode quality set against the reference and LeWM-Flash on the same GPU. The H100 reference and LeWM-Flash quality runs used in the ladder were made in this phase.
- **Phase 2, the ladder (17 October 2026).** A first batch-1 pass of R0 to R3 and R3-CEM (Phase A) and quality runs of R1 and R2; then the fused encoder, its parity check, and a final batch-1 pass of all seven rungs with quality runs of R3, R3-CEM and R4.

Three deviations from the plan are material. The study was originally budgeted at $10 of GPU time for a megakernel on its own; the user extended it to $35 when the equal-hardware ladder was added, and the programme used about $31.50 (Appendix D). The batch-1 harness starts a new process per seed and pays the world construction, model load and, for

R2b, the compile warmup 61 times, which made each latency pass cost $10 to $12 and limited the ladder to two passes. And the quality runs of the seven rungs were made in three batches over two days rather than in one, which is harmless to the pairing because the episodes are fixed by seed, but means the 300-episode results of R0 and R2b come from the Phase 1 runs with the same flags rather than from the ladder's own invocation.

2.8 Implementation and infrastructure

The planner code is the LeWM and stable-worldmodel source with the LeWM-Flash solvers (lewm/solvers/) and the benchmark harness (tools/benchmarks/lewm_plan_benchmark.py). The megakernel is a single CUDA source (megakernel/lewm_mk.cu, about 1,600 lines) with PyTorch bindings, a weight folder and panel packer (pack.py), a runtime wrapper (runtime.py) and a solver class that plugs into the harness (solver.py). Builds are content-addressed by the hash of the source, so a container never runs a stale extension. All GPU work ran on Modal: kernel builds on CPU containers with nvcc 12.8, and parity checks, micro-benchmarks, ladder passes and quality runs on single H100 containers, with each run's metrics, manifest and logs written to a results volume and committed to the repository. The analysis script docs/paper/src/compute_stats.py regenerates every latency and success statistic in this paper from the committed per-call records (Appendix E).

3. Results

We report the ladder in the order of its rungs, because each rung sets the terms of the next. Section 3.1 gives the end-to-end result, Sections 3.2 to 3.5 the contribution of each step and its mechanism, Section 3.6 the paired closed-loop evidence that no step changed task success, Section 3.7 the numerical agreement of the megakernel with the reference, and Section 3.8 how the kernel reached its final latency and what did not work. Every latency in this chapter is a batch-1 end-to-end plan latency on the same H100 in the same container image unless it is labelled otherwise, and every success rate is on the same 300 paired Push-T episodes.

3.1 The ladder end to end

Table 4 and Figure 4 give the result. The reference LeWM planner takes 622 ms to produce a plan for one robot on an H100. The final rung, one megakernel launch from the raw observation to the action distribution, takes 46.5 ms, 13.4 times less, with a p90 of 47.0 ms. The best PyTorch implementation, LeWM-Flash, takes 182 ms; the megakernel rungs are 3.1 and 3.9 times faster than it. Task success is 260 to 265 of 300 on every rung.

Table 4. Batch-1 end-to-end plan latency on one H100 (final pass; 72 to 76 plans per rung, first seed dropped) and Push-T success on 300 paired episodes. Step gain is the ratio of the medians of a rung and the rung above it; R2b is compared with R2, R3 with R2, R3-CEM with R1.

rung	plan p50 (ms) [95% CI]	p90 (ms)	step call p50 (ms)	step gain	total vs R0	success (Wilson 95%)
R0 reference LeWM (CEM, FP32 eager)	622.4 [605.5, 626.7]	673.8	2.14	—	1.0×	261/300 (82.7–90.3%)
R1 + cached embeddings, no host sync	377.0 [375.8, 381.0]	401.0	2.02	1.65×	1.65×	263/300 (83.5–90.9%)
R2 + iCEM, FP32 eager	410.2 [406.2, 416.3]	433.3	2.11	0.92×	1.52×	265/300 (84.2–91.5%)
R2b LeWM-Flash (compile, TF32, BF16)	182.1 [177.3, 185.7]	204.3	1.51	2.25×	3.42×	261/300 (82.7–90.3%)
R3 + megakernel (predictor + iCEM)	59.3 [59.1, 59.4]	61.1	2.35	6.92×	10.5×	262/300 (83.1–90.6%)
R4 + fused encoder, one launch	**46.5 [46.4, 46.6]**	**47.0**	**0.82**	**1.28×**	**13.4×**	260/300 (82.4–90.1%)
R3-CEM megakernel with CEM 300 × 30	82.1 [81.7, 82.5]	83.6	1.59	4.59×	7.58×	261/300 (82.7–90.3%)

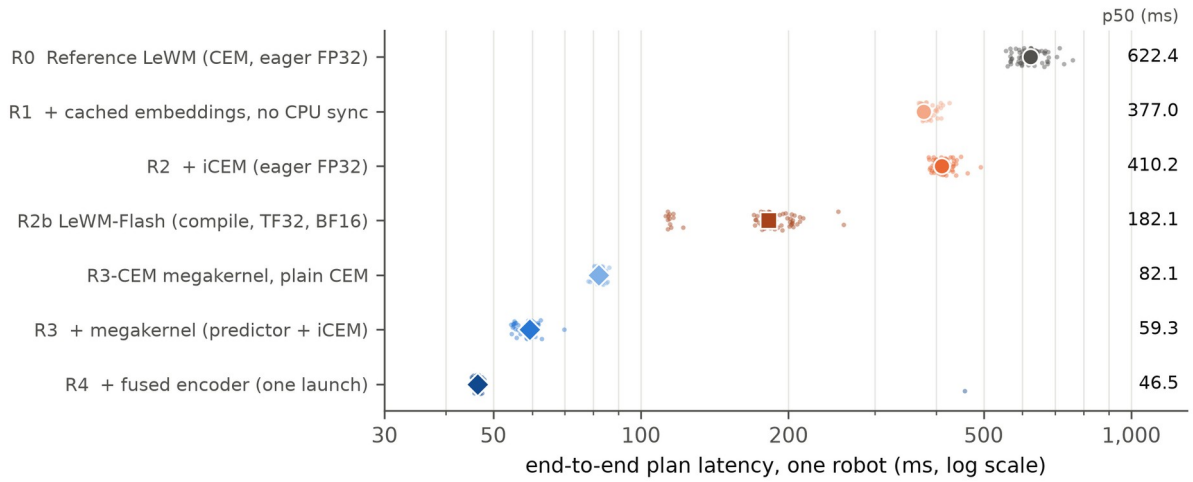


Figure 4. Batch-1 end-to-end plan latency per rung on one H100, log scale. Small points are individual plans (72 to 76 per rung); large markers are medians, also given at the right. Orange rungs are PyTorch, blue rungs are the megakernel.

The shape of the distributions matters as much as their medians for a controller. The PyTorch rungs spread over tens to hundreds of milliseconds: R0 from 556 to 759 ms, R2b from 112 to 259 ms with two distinct clusters. The megakernel rungs are tight: R3 spans 54 to 70 ms, and R4's interquartile range is 46.1 to 46.8 ms. R4 has one outlier, a single plan of 458 ms among 76, which raises its mean to 51.8 ms; it is a one-off in its seed's process and we did not isolate its cause. The step calls, which only pop a buffered action, fall from about 2 ms to 0.82 ms at R4 because the fused path no longer runs the policy's CPU image transform on every call.

3.2 Caching and synchronisation: 1.65×, exact

Encoding the observation and goal once per plan instead of in every one of the 30 cost calls, and keeping the CEM loop on the GPU, cuts the plan from 622 to 377 ms. The change is exact: on fixed candidates it reproduces the reference costs and rankings with zero difference (LeWM-Flash exactness check). On the 300 paired episodes it differs from the reference in two episodes, both successes for R1 (263 against 261; McNemar p = 0.50); because the costs are identical on fixed candidates, the two must come from inputs that check does not fix. The most plausible source is floating-point differences of the order of 10^{-7} between the two code paths' encoder and cost calls, whose tensors have different shapes and layouts, which can flip a near-tie in a ranking and send a sensitive episode down a different path.

The remaining 377 ms is almost entirely the rollout: 30 sequential cost calls, each a 5-step autoregressive pass through the predictor for 300 candidates, issued one PyTorch operator at a time.

3.3 iCEM: no gain in eager PyTorch, 1.38× in the megakernel

iCEM evaluates 3.19 times fewer candidate rollouts per plan than CEM. In eager PyTorch it makes the plan slower, 410 ms against 377 (0.92×), with success unchanged (265 against 263 of 300; 17 against 15 discordant episodes, p = 0.86). Figure 5 shows why. A single eager FP32 cost call takes 20 ms whether it evaluates 60 candidates or 300: the call is a sequence of a few hundred small kernels, each launched from Python, and at these sizes

the GPU finishes each kernel before the next is issued, so the time is set by the number of launches and not by the number of candidates. Reducing the population therefore buys nothing, and iCEM's small extra work per iteration (retaining elites, a variable population) costs a little. In this micro-benchmark the eager solve takes 601 ms, 30 calls of about 20 ms; the same path inside the benchmark harness is faster in absolute terms (Table 4), a difference we did not isolate, but it is equally flat in the population.

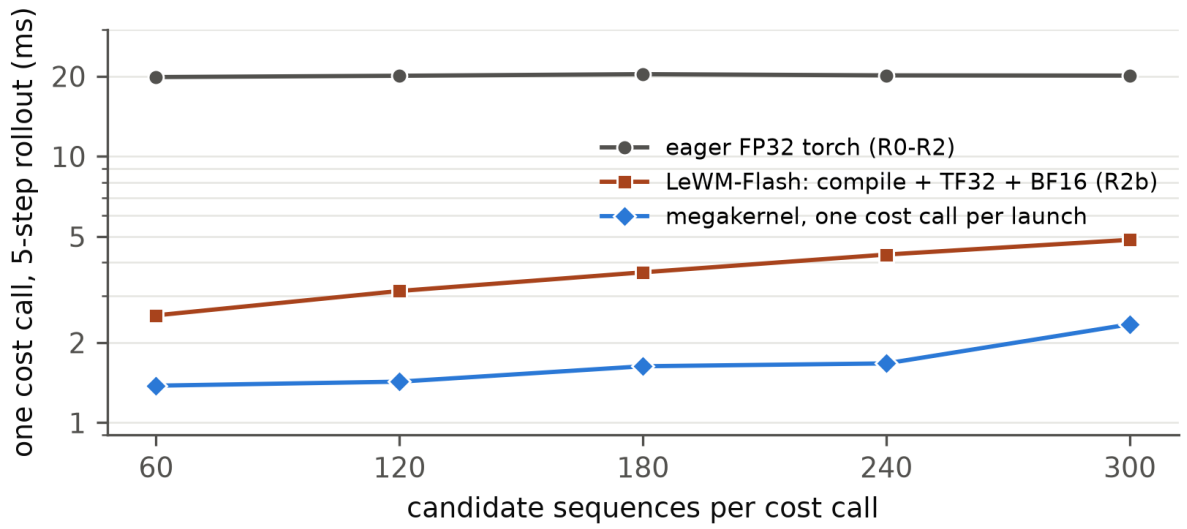


Figure 5. Latency of one cost call (a 5-step rollout of every candidate through the predictor) against the number of candidates, on one H100. Eager FP32 PyTorch is flat at about 20 ms; LeWM-Flash's compiled BF16 path grows from 2.5 to 4.9 ms; the megakernel, timed here as a launch per cost call, from 1.4 to 2.3 ms. Micro-benchmark with cached embeddings; medians of 30 to 50 calls.

Once execution stops being launch-bound, the population size matters. LeWM-Flash's compiled cost call grows from 2.5 ms at 60 candidates to 4.9 ms at 300, and the megakernel's from 1.4 to 2.3 ms. The clean measure of iCEM's value is the side rung R3-CEM, the same megakernel running the reference CEM schedule: 82.1 ms against R3's 59.3 ms, a 1.38 times gain, with success unchanged (262 against 261; 20 against 19 discordant, p = 1.00). The gain is smaller than the 3.19 times reduction in evaluations because the predictor chain of a plan has 30 iterations regardless of population, and at 60 candidates the 64-row tiles of the kernel are almost empty: the cost per iteration falls by less than the population does. iCEM is a planner-level change whose payoff depends on the execution model beneath it, which is the first finding of the ladder.

3.4 The megakernel: 6.9× over eager iCEM, 3.1× over LeWM-Flash

R3 runs exactly R2's planner, with the same population schedule, elites and horizon, as one kernel launch per plan, and takes 59.3 ms against R2's 410 ms (6.92×) and LeWM-Flash's 182 ms (3.07×). Its single-environment solve, the part of the plan inside the launch, takes 41.3 ms in isolation, against 82.3 ms for LeWM-Flash's compiled BF16 solve and 601 ms for eager FP32 in the same micro-benchmark. The remaining 18 ms of R3's plan is outside the kernel (Figure 6): the PyTorch FP32 encoder for the observation and goal (6.9 ms) and the policy's CPU image transform and glue (about 11 ms by difference).

The megakernel's advantage over LeWM-Flash is not arithmetic. Both use BF16 tensor cores with FP32 accumulation, and both do the same 2,820 rollouts. What the megakernel removes is everything between the arithmetic: the 30 host round trips of the iCEM loop, the

per-operator launches and the graph replay of the compiled path, the materialisation of every intermediate activation in global memory, and the reloading of the weights for every operator. Inside the kernel a plan is a chain of 900 dependent layer evaluations, each of about twenty sub-microsecond steps, with weights streamed from L2 into shared memory ahead of use and activations never leaving the chip between layers.

3.5 The fused encoder: 1.28×, one launch per plan

R4 moves the encoder into the same launch, reads the raw uint8 frame, and caches the goal embedding for the episode. The plan falls from 59.3 to 46.5 ms (1.28×). In isolation, encoding the observation and goal takes 1.34 ms in the kernel (1.25 ms for the observation alone, when the goal is cached) against 6.9 ms for the FP32 PyTorch encoder. The larger part of the gain, however, comes from what fusion removes around the encoder: R4 skips the policy's CPU image transform, which also removes most of the step-call latency, and the glue between plan and kernel shrinks from about 11 ms to about 4 ms (Figure 6). Of R4's 46.5 ms, about 41.3 ms is the predictor and iCEM solve, 1.3 ms the encoder and 3.9 ms everything else.

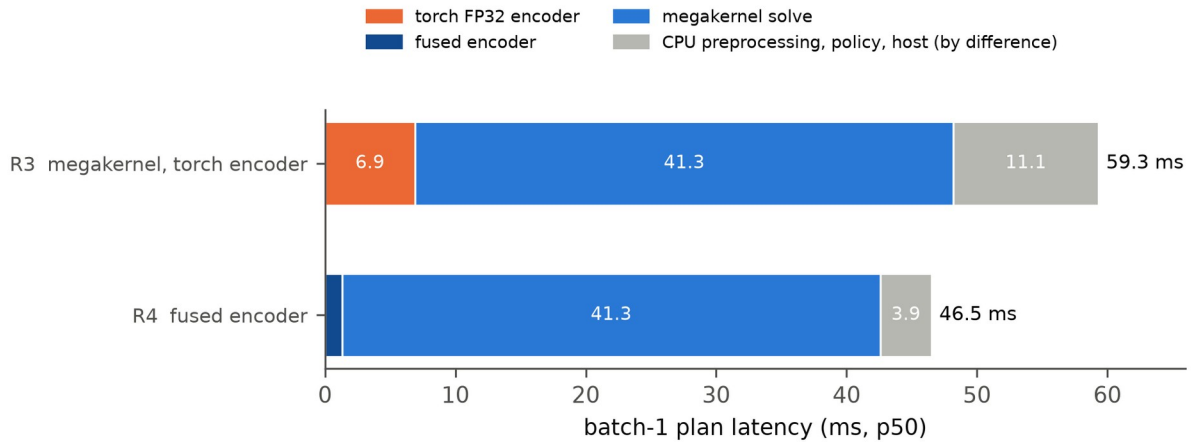


Figure 6. Composition of the R3 and R4 plans (medians). The solve and encoder times are isolated measurements of the same kernel and encoders on the same GPU; the remainder (grey) is the difference to the end-to-end median and contains the policy's CPU preprocessing, Python glue and the transfers to and from the host.

3.6 Task success is unchanged at every rung

The ladder's claim on accuracy is a null result, and Table 5 and Figures 7 and 8 give its evidence. Every rung's success rate lies between 260 and 265 of 300, every Wilson interval overlaps every other, and no paired comparison with the reference or between adjacent rungs comes close to significance (smallest McNemar p = 0.50). The final rung differs from the reference in 39 of 300 episodes, 19 won and 20 lost, for a paired difference of −0.3 points with a 95% interval of −4.3 to +3.7.

Table 5. Paired comparisons on the 300 Push-T episodes. Discordant episodes are counted as wins for the first rung / wins for the second; the interval is a paired bootstrap over episodes (10,000 resamples); p is the exact McNemar test.

comparison	what changes	difference (points) [95% CI]	discordant	p
R1 vs R0	caching, no host sync	+0.7 [0.0, +1.7]	2 / 0	0.50
R2 vs R1	CEM → iCEM	+0.7 [−3.0, +4.3]	17 / 15	0.86

comparison	what changes	difference (points) [95% CI]	discordant	p
R2b vs R2	compile, TF32, BF16	−1.3 [−4.3, +1.7]	8 / 12	0.50
R3 vs R2	megakernel (BF16, Philox)	−1.0 [−4.7, +2.3]	13 / 16	0.71
R3 vs R2b	megakernel vs LeWM-Flash	+0.3 [−3.3, +4.0]	16 / 15	1.00
R3-CEM vs R1	megakernel, CEM schedule	−0.7 [−4.3, +3.0]	15 / 17	0.86
R3 vs R3-CEM	iCEM inside the megakernel	+0.3 [−3.7, +4.3]	20 / 19	1.00
R4 vs R3	fused encoder	−0.7 [−4.0, +2.7]	12 / 14	0.85
R4 vs R0	the whole ladder	−0.3 [−4.3, +3.7]	19 / 20	1.00

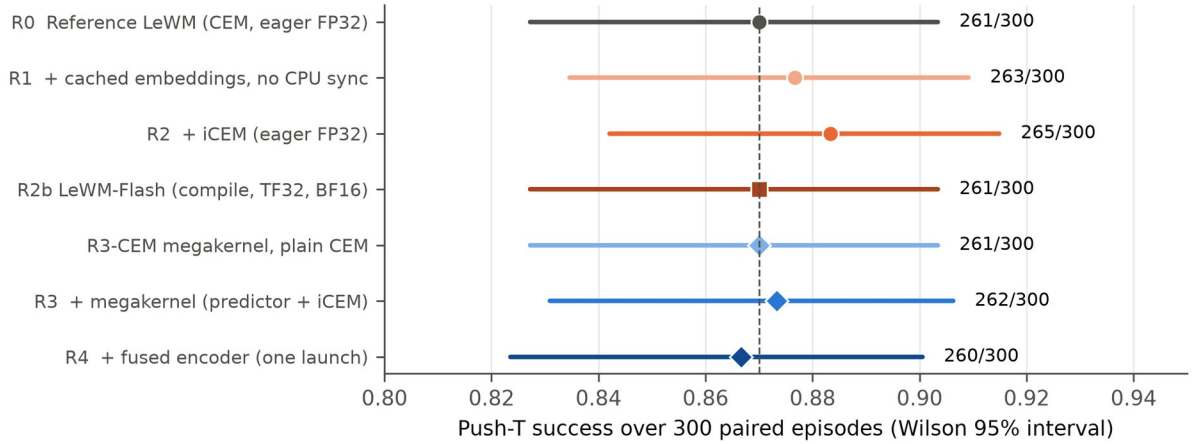


Figure 7. Push-T success per rung on the 300 paired episodes, with Wilson 95% intervals. The dashed line is the reference.

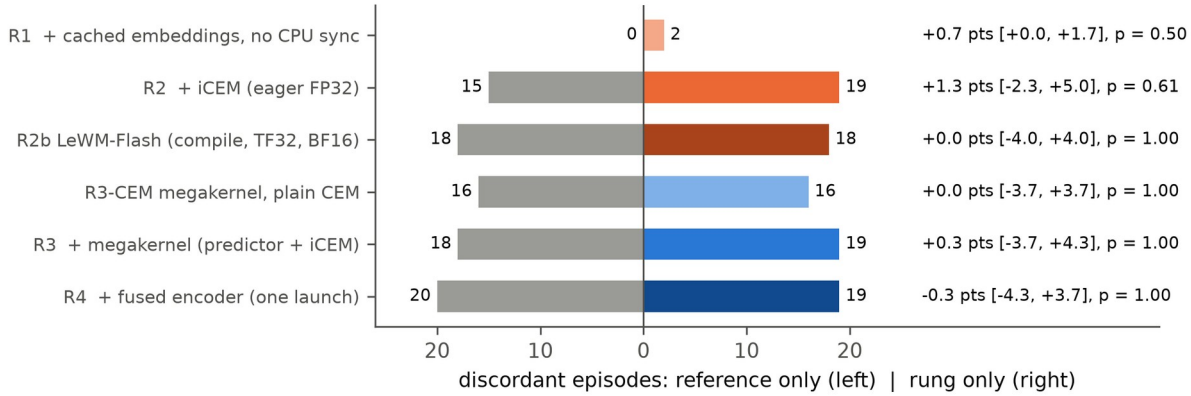


Figure 8. Paired outcomes against the reference. Each row counts the episodes that only the reference solved (left, grey) and that only the rung solved (right). The annotation gives the paired difference, its bootstrap 95% interval and the exact McNemar p-value.

Two features of the paired data deserve comment. The exact rung, R1, differs from the reference in 2 episodes, while every rung that changes the optimiser, the numerics or the random stream differs from it in 32 to 39. That is the scale of the variation a change of sampling noise alone produces in this task: an episode near the success boundary is decided by the particular candidates drawn, and a new random stream redraws them. The discordance between R3 and R3-CEM (39 episodes) is as large as that between R4 and the reference, although the two share their kernel and numerics and differ only in the population

schedule. The comparisons therefore resolve differences of about 6 points at conventional power (Section 2.6.3); within that resolution, no step of the ladder changed what the planner achieves.

3.7 Numerical agreement with the reference

The megakernel is a numeric change, and Table 6 places its error against the accepted BF16 autocast path of LeWM-Flash on identical inputs. On eight (problem, population, variance) cases, the maximum relative cost error against FP32 is 0.0005 to 0.0043 for the megakernel and 0.0005 to 0.0040 for BF16 autocast, and both recover 27 to 30 of the FP32 top-30 candidates. The fused encoder is closer to FP32 than PyTorch's own BF16 autocast: its embeddings deviate by 0.41% in relative L2 against 0.52%, with a minimum cosine similarity of 0.99999, and costs computed from its embeddings are within 0.32% of FP32 with identical top-30 sets.

Table 6. Numerical agreement with the FP32 eager reference on identical inputs (H100). Cost error is the maximum relative error over all candidates of a case; top-30 is the overlap with the FP32 top-30 set.

quantity	LeWM-Flash BF16 autocast	megakernel
predictor cost, max relative error (8 cases)	0.0005 – 0.0040	0.0005 – 0.0043
top-30 overlap with FP32 (8 cases)	28 – 30	27 – 30
batched launch, 6 environments: max relative error / top-30	—	≤ 0.0029 / 29 – 30
encoder embedding, relative L2 / minimum cosine (32 frames)	0.0052 / 0.999986	0.0041 / 0.999992
cost from encoder embeddings, max relative error / top-30 (6 cases)	—	≤ 0.0032 / 30

Planner-level agreement was tested separately: 16 (observation, goal) problems were solved by the megakernel in one batched launch and by the PyTorch iCEM solver, and each returned plan was re-scored by the FP32 model. The median plan cost was 189.23 for the megakernel and 189.31 for PyTorch, and the megakernel's plan was better or equal on 7 of 16. The plans differ draw for draw, because the noise streams differ, but they come from the same distribution.

3.8 How the kernel reached 41 ms, and what did not work

The single-environment solve fell from 76.9 ms for the first correct kernel to 41.3 ms in five steps (Figure 9, Table 7). Each step removed a latency term from the dependent chain rather than arithmetic.

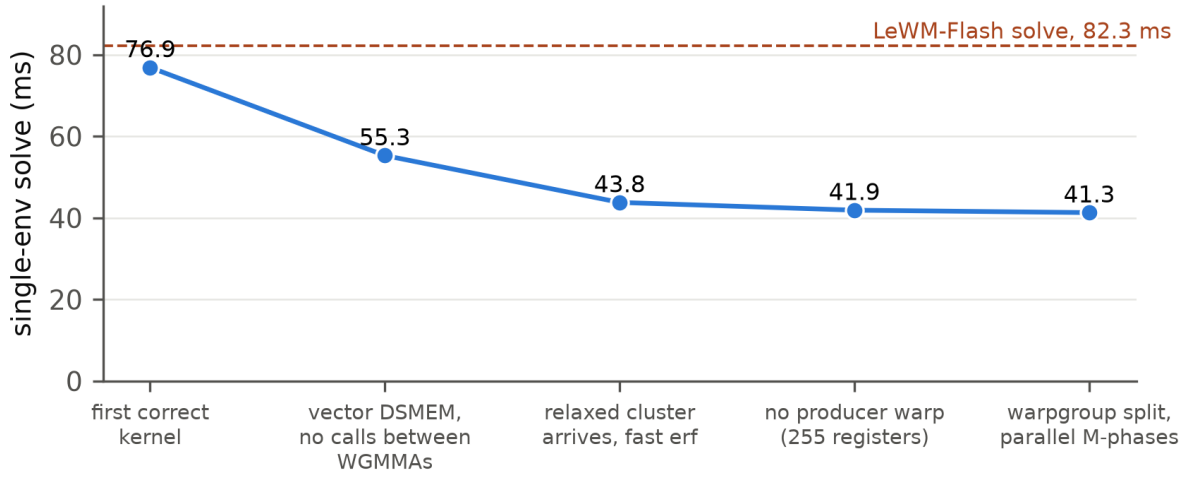


Figure 9. Single-environment solve time of the megakernel through its development (H100, CUDA events, 30 iCEM iterations with the bucketed schedule). The dashed line is LeWM-Flash's compiled BF16 solve in the same micro-benchmark.

Table 7. Megakernel tuning log (single-environment solve). Accepted steps in order, then reverted attempts.

step	solve (ms)	what changed and why
first correct kernel	76.9	parity with FP32 at the first run; the compiler had serialised every WGMMA because a `printf` failure path was reachable from the GEMM loop
vectorised data movement	55.3	16-byte DSMEM, shared-memory and AdaLN-table accesses; an inline trap instead of a function call on the failure path, so nothing interrupts the WGMMA chains
relaxed cluster barrier, fast erf	43.8	one release fence and relaxed remote arrives: cluster barrier 3.6 → 0.65 µs; the library `erff` and reciprocal replaced by the Abramowitz–Stegun form with a fast divide
no producer warp	41.9	each warpgroup refills its own weight ring; eight warps keep 255 registers per thread, a ninth caps them at 168 and spills
warpgroup split at 16-CTA clusters, parallel M-phases	41.3	at batch 1 the two warpgroups of a CTA split q, k and the output projection from v; action-encoder and AdaLN phases split over (row block, slice) units with atomic accumulation
reverted: peel the first accumulate of each GEMM chain	worse	register pressure rose and spills grew
reverted: prefetch cached K and V with the weights	worse	as above
reverted: prefetch row-owner operands across the cluster barrier	worse	as above

The micro-benchmarks bound what remains. A cluster barrier costs 0.65 µs and a grid barrier 1.6 µs; a 64 × 64 epilogue 0.15 µs; an empty launch 14 µs. Each of the 900 layer evaluations of a plan passes four cluster barriers and two row-owner reduction rounds, and each iteration four grid barriers; those synchronisation costs alone account for several milliseconds, and the rest of the 41 ms is the latency, not the throughput, of the WGMMA chains and their epilogues at 64-row tiles. The weight ring almost never waits. Getting much

below 40 ms at batch 1 would need fewer cross-SM hand-offs per layer, for example one layer per cluster with resident weights or an all-reduce that removes the owner round trip, which is a redesign rather than a tuning step.

The ladder was measured twice (Appendix C). The megakernel rungs reproduced to within 1.3 ms (R3 58.0 and 59.3 ms; R3-CEM 81.7 and 82.1 ms). The PyTorch rungs moved by up to 18%: the reference from 527 to 622 ms, R1 from 391 to 377, R2b from 196 to 182, with R2 unchanged at 410. Launch-bound execution inherits the variance of the host CPU and of the container it lands on; a single launch does not.

4. Discussion

4.1 Answers to the research questions

RQ1. How much of the reference planner's latency is redundant work that an exact change removes? About 40%. Encoding the observation and goal once per plan and keeping the loop on the GPU takes the reference from 622 to 377 ms at batch 1, with identical costs on fixed candidates and identical success to within two episodes. This is the cheapest gain of the ladder and the one any deployment of a JEPA planner should take first, because the released planner treats the encoder as part of the cost function and pays for it in every optimiser iteration.

RQ2. Does iCEM make planning faster, and does it change task success? It depends on the execution model, and it does not change success. In eager PyTorch iCEM is 8% slower than CEM although it evaluates 3.19 times fewer rollouts, because each cost call is launch-bound and costs the same at 60 candidates as at 300. In the megakernel it is 1.38 times faster than CEM on the same kernel. In both, its success on 300 paired episodes is within a point of CEM's (265 against 263 eager; 262 against 261 in the kernel). A sample-efficient planner is only as fast as the cost of its smallest batch, and the ladder's ordering matters for its accounting: had iCEM been measured only in eager PyTorch it would have been rejected, and had it been measured only on top of the megakernel its contribution would have been read as a third of what the evaluation count suggests.

RQ3. How much does executing the whole planner as one persistent kernel gain over the best PyTorch implementation of the same planner? A factor of 3.1 end to end (182 to 59 ms), and 2.0 on the solve alone (82.3 to 41.3 ms), at the same numerics class and the same number of rollouts. Against the eager implementation of the same planner it is 6.9 times. The megakernel gains by removing the latency between arithmetic, not the arithmetic: host round trips, launches and graph replays, activation traffic through global memory, and per-operator weight reloads.

RQ4. Do the numeric changes of the megakernel and the fused encoder preserve the planner's accuracy? Yes, within the resolution of the evaluation. The kernel's cost error against FP32 is the same as that of PyTorch's BF16 autocast, the fused encoder is closer to FP32 than BF16 autocast is, and on 300 paired episodes the final rung differs from the reference by −0.3 points (95% CI −4.3 to +3.7; 19 against 20 discordant). We do not claim the planner is unchanged to better than about 6 points of paired difference, which is what 300 episodes can resolve; we claim that no step of the ladder produced a detectable change, and that the variation between rungs is of the size that a new random stream alone produces.

4.2 Why a JEPA planner is a latency problem, not a throughput problem

The workload of a JEPA planner is unusual among neural-network inference workloads, and the ladder's results follow from its shape. The model is tiny and the computation is deep. One plan is 30 optimiser iterations, each a 5-step autoregressive rollout through 6 transformer layers, for 60 to 300 candidates that together form matrices of 64 to 300 rows. The arithmetic of a whole plan is a few hundred GFLOP, which an H100's tensor cores finish in well under a millisecond, and the weights fit in L2. Neither of the two quantities by which inference is usually optimised, FLOPs and bytes moved, is binding. What binds is the length of the dependent chain, 900 layer evaluations per plan, and the fixed cost of each link.

Each rung of the ladder attacks a different term of that fixed cost. Caching removes work from the chain. iCEM shortens each link's width but not the chain's length, which is why its gain is bounded by how much a link's cost depends on its width. Compilation and CUDA graphs, in LeWM-Flash, shorten the per-operator cost of each link but keep the per-iteration host round trip and the global-memory traffic between operators. The megakernel removes both: a link is now a WGMMA chain on data already in shared memory, followed by a cluster barrier. What remains is the hardware's own latency for small dependent operations, measured in Section 3.8 in fractions of a microsecond per barrier and epilogue.

This has a practical consequence for how JEPA world models are deployed. The usual levers of inference optimisation, lower precision and larger batches, help little at batch 1: BF16 roughly halves the arithmetic that was not binding, and batching amortises costs a single robot cannot share. The lever that matters is fusion across the whole planner, and the planner's structure makes that fusion natural: its loop has a static shape once the population schedule is fixed, so the weights it needs, the barriers it crosses and the memory it touches are all known before it starts. That static shape is what allows a weight stream with no producer warp and a schedule with no host involvement.

4.3 What the megakernel design needed

Three decisions carried most of the gain, and each has a counterpart in the megakernel literature that we can now confirm for this workload class.

Weights as a precomputed stream rather than a cache. Because the planner's sequence of operations is fixed, the panel sequence is known in advance, and each warpgroup can prefetch its next weights the moment it releases a buffer, through any boundary. The weight ring almost never waited. The design choice that made this possible without a producer warp was not an algorithm but a register budget: Hopper's register file gives 255 registers per thread to eight warps and 168 to nine, and every attempt that raised register pressure (peeling, prefetching K and V, prefetching row-owner operands) made the kernel slower through spills.

Cheap cluster synchronisation. Tensor parallelism across a cluster puts four cluster barriers in every layer, so the barrier is on the critical path 3,600 times per plan. A barrier with a release on every remote arrive cost 3.6 µs; one fence and relaxed arrives cost 0.65 µs. That single change was worth about 11 ms of the solve (55.3 to 43.8 ms, together with the fast erf). The distributed shared memory of thread-block clusters is what makes tensor parallelism inside a kernel viable at all, but its synchronisation primitives have to be used with the memory ordering they need and no more.

Numerics chosen to match an accepted path, not to minimise error. The kernel uses BF16 operands with FP32 accumulation, residual, LayerNorm statistics and costs, because that is the precision class whose closed-loop accuracy the LeWM-Flash study had already accepted. Matching an accepted class let numerical acceptance be a comparison of error distributions on identical inputs rather than a new closed-loop question for every kernel change. The same choice explains why the fused encoder could be adopted quickly: its error is below that of the BF16 autocast path it replaces.

4.4 Position relative to related methods

Table 8 sets the ladder against the approaches to fast planning and fast small-model inference that bear on it. The comparison is on what each approach changes and what it needs, because the latency numbers are on different models, tasks and hardware and are not comparable at the precision that would matter.

Table 8. The ladder against related approaches to fast world-model planning and small-model inference.

approach	what it changes	planner preserved	numerics	execution	batch-1 end-to-end evidence
LeWM as released (Maes, Le Lidec, Scieur, et al., 2026)	—	CEM 300 × 30	FP32	eager PyTorch	planning time reported per plan
iCEM (Pinneri et al., 2021)	sample-efficient CEM: decaying population, elite reuse, coloured noise	no (planner change)	unchanged	unchanged	fewer evaluations; wall time depends on execution
model change: parallel prefix prediction (Gao & Xu, 2026)	retrains the predictor to roll out all action prefixes in one pass	yes (CEM)	unchanged	eager PyTorch	about 2× less CEM time, retraining required
amortised or gradient-based planning (Nguyen et al., 2026; Y. Wang et al., 2026)	replaces the sampling planner with a learned inverse-dynamics policy or gradient descent	no	unchanged	unchanged	100–130× less per decision (amortised); not a like-for-like planner
GPU sampling MPC (Vlahov et al., 2024)	fuses a rollout's dynamics and cost into one kernel, intermediates in shared memory	yes (MPPI)	FP32	combined or split CUDA kernels	millisecond MPC with analytic dynamics, no neural model
compiler and graph capture (`torch.compile`, CUDA graphs; LeWM-Flash)	per-operator overhead	yes	TF32 / BF16	many kernels, host loop	182 ms (this paper, R2b)
megakernels for LLM decoding (Spector, Juravsky, Sul, Dugan, et al., 2025; Nrusimha et al., 2025; Cheng et al., 2026)	whole forward pass in one launch	— (no planner)	BF16	one launch per token	token latency on large models
this paper (R4)	whole planner, encoder to action distribution, one launch	yes (iCEM of R2)	BF16 operands, FP32 accumulation	one launch per plan	46.5 ms, success unchanged on 300 paired episodes

Two properties distinguish the ladder. Among the entries that apply to a neural world model, it is the only one that keeps the model, its weights and the planner fixed, down to the population schedule, elite count and horizon, and still reaches the time scale of control; the other routes buy their gains with retraining, a different optimiser or a learned amortisation, and they compose with ours rather than compete with it. And it is the only one whose fused unit is an entire optimisation loop rather than a single forward pass: the megakernel for LLM decoding fuses one model evaluation per launch, while a planning call is 2,820 evaluations, 30 synchronising reductions and their distribution updates. Against that, it is specific to one model architecture and one GPU generation, and its engineering cost is not comparable to switching on a compiler.

4.5 Limitations and threats to validity

Statistical resolution of the accuracy claim. Three hundred paired episodes resolve paired differences of about 6 points at conventional power. Every claim that a rung preserves task success is a claim of no detectable change at that resolution. The rungs that change the random stream differ from the reference in 32 to 39 episodes each, so a systematic difference of 2 or 3 points in either direction would not be visible; resolving one would need about 2,400 pairs.

Latency sample and variance. Each rung's latency rests on 72 to 76 plans from one process per seed, on one GPU of one cloud provider, measured in two passes. The PyTorch rungs moved by up to 18% between the passes, so their medians are best read as typical rather than exact, and the step gains involving them (1.65× for caching, 0.92× for iCEM in eager mode, 3.1× for the megakernel over LeWM-Flash) carry that uncertainty. The megakernel rungs reproduced to within about 2%. The R2b distribution is bimodal and R4 has one 458 ms outlier; we report both and did not isolate either cause.

Micro-benchmark and harness disagree in absolute terms. The isolated eager FP32 solve (601 ms) is slower than the eager rungs' end-to-end plans in the harness (377 to 410 ms), and the composition of Figure 6 combines isolated measurements with an end-to-end median by difference. The conclusions drawn from the micro-benchmarks depend on their shape (flat latency in the population for eager execution; the ordering of the three paths), not on their absolute level, but the decomposition of R3 and R4 is an attribution rather than a measurement.

One model, one task, one GPU. All results are for LeWM's Push-T checkpoint on an H100. The kernel's tile shapes, cluster sizes and panel layout are written for this architecture's dimensions (width 192, 16 heads of 64, MLP 2,048, history 3) and for Hopper's WGMMA, clusters and DSMEM; a different JEPA, a longer horizon or a Blackwell GPU would need retuning, and the kernel does not run on the L40S on which LeWM-Flash was developed. The mechanism, a latency-bound chain that fusion shortens, is a property of sampling-based planning with small world models, and we expect the ordering of the ladder to transfer; the factors are specific to this system.

Evaluation batches and reused runs. The quality runs were made in three batches over two days, and the reference and LeWM-Flash runs come from the megakernel phase with the ladder's flags rather than from the ladder's own invocation. Episodes are fixed by seed, so the pairing is exact, but the runs were not interleaved, and container-level differences between days cannot be excluded for the PyTorch rungs' latency. They do not affect success, which does not depend on timing.

Simulation only, and only the planner. Push-T in simulation is the benchmark on which LeWM reports its planning results, and a 46.5 ms plan is a claim about compute, not about control. A real robot adds sensing and actuation latency, and the control rate it needs depends on the task. We also optimised only the planning call; the 48 non-planning calls of an episode are measured but were optimised only incidentally, by the fused path's removal of the per-call image transform.

Budget. The programme ran on rented GPU time against a fixed budget and used about $31.50 of the $35 approved (Appendix D). That bounded the ladder to two latency passes, one quality evaluation per rung and no repeated quality seeds, and it is why the latency of the batched evaluation regime is reported only as an appendix.

5. Future work

The results point to five lines of work, in the order we would pursue them.

Spend the saved latency on planning. The ladder held the planner's budget fixed in order to measure its execution, but a deployed planner would choose its budget against its deadline. At 46.5 ms a plan with twice the iterations or the full 300-candidate population in every iteration still fits a 100 ms control period, and the megakernel prices those choices exactly: R3-CEM shows that the full CEM budget costs 23 ms more than iCEM in the kernel. The LeWM-Flash telemetry showed iCEM's elite cost still falling between iterations 25 and 30. A sweep of iterations, population floor and retained elites on the 300 paired episodes, now affordable at a few milliseconds per plan, would map the latency-success frontier of the planner rather than one point on it, and would test whether the 9 points between our reference's 87% and LeWM's reported 96% are within reach of planning budget alone.

Fewer hand-offs per layer. The kernel's remaining 41 ms is the latency of a 900-layer dependent chain with four cluster barriers and two row-owner rounds per layer, and the tuning log shows that further register-level work makes it slower. The next step is structural: an all-reduce that removes the owner round trip, or a pipeline in which each cluster holds one layer's weights resident, in FP8 if necessary, and candidates stream through clusters. Either would trade the tensor-parallel schedule's barriers for point-to-point hand-offs, and the primitive costs measured here (0.65 µs per cluster barrier, 1.6 µs per grid barrier) are enough to estimate the gain before writing it.

Compose with model-level speedups. The model and planner were held fixed by design, and the routes that change them are complementary. Fast LeWorldModel's parallel prefix prediction would shorten the chain from five predictor passes per iteration to one (Gao & Xu, 2026), which inside the megakernel would remove most of the dependent depth that sets its latency; an amortised inverse-dynamics policy (Nguyen et al., 2026) could warm-start the sampling distribution and allow fewer iterations. Both would need their own paired evaluation, and the ladder's protocol is ready for them.

Other world models and other hardware. LeWM's single-token representation makes it the most favourable case for a megakernel; DINO-WM's patch tokens and V-JEPA 2-AC's larger predictor make the planner's matrices larger and push the workload back towards throughput, where the case for fusing the whole optimiser loop should be measured rather than assumed. On the hardware side, the kernel is written for Hopper; Blackwell's tensor-memory and larger clusters, and onboard computers such as the Jetson Thor, change both the primitive costs and the arithmetic budget. An automated megakernel compiler that supports the shape dynamism of iCEM (Cheng et al., 2026; Jin et al., 2026) would make that porting a compilation rather than a rewrite, and LeWM's planner is a compact test case for one.

Evaluate at the resolution a deployment needs, and on a robot. Three hundred paired episodes exclude changes of about 6 points in task success and no smaller. A deployment that cares about 2 points needs about 2,400 paired episodes per comparison, which at the megakernel's speed is now an evaluation of minutes rather than hours, and a sequential paired test would let it stop early. The final step is a closed loop on a real robot, where the question becomes not how fast a plan is computed but how fast the planner needs to be for the task, and whether replanning at every control step, which a 46.5 ms plan permits and the released 622 ms plan did not, improves what the robot achieves.

6. Conclusion

We set out to measure how far the planning latency of a JEPA world model can be reduced without changing the model, the planner's semantics or its task success, and which kinds of change account for the reduction. We built a seven-rung ladder on one H100 from the released LeWM planner, adding one change per rung, and evaluated every rung for the end-to-end latency of one replanning call for one robot and for closed-loop success on the same 300 Push-T episodes.

The answer has three parts. Exact changes, encoding the observation and goal once per plan and keeping the optimiser on the GPU, remove 40% of the reference latency (622 to 377 ms) at no cost to anything. The improved cross-entropy method, which evaluates 3.19 times fewer candidate rollouts, is worth nothing in eager PyTorch, where every cost call is launch-bound and costs the same at 60 candidates as at 300, and 1.38 times inside a megakernel; its value is a property of the execution model as much as of the algorithm. And executing the whole planner, encoder, predictor rollouts, cost, elite selection and distribution updates for all 30 iterations, as one persistent ThunderKittens kernel takes a plan to 46.5 ms with a p90 of 47.0 ms: 13.4 times faster than the reference and 3.9 times faster than the best PyTorch implementation of the same planner, with BF16 operands and FP32 accumulation whose error matches the accepted autocast path and whose closed-loop success is indistinguishable from the reference (−0.3 points, 95% CI −4.3 to +3.7).

The mechanism is the shape of the workload. A JEPA planner is a small model applied in a long chain of dependent steps, and at batch 1 it is bound neither by arithmetic nor by memory bandwidth but by the latency of each link. The levers that matter are the ones that shorten that chain or the fixed cost of its links, and the one that matters most is to stop leaving the GPU between them.

LeWM made world-model planning fast enough to discuss for control by shrinking the model. This paper shows that the execution of the planner, not its mathematics, was most of what remained, and that removing it takes JEPA planning from the time scale of deliberation to the time scale of control on the hardware where these models are trained.

References

Abramowitz, M., & Stegun, I. A. (1964). *Handbook of mathematical functions with formulas, graphs, and mathematical tables* (National Bureau of Standards Applied Mathematics Series 55). U.S. Government Printing Office.

Andersch, M., Palmer, G., Krashinsky, R., Stam, N., Mehta, V., Brito, G., & Ramaswamy, S. (2022, March 22). *NVIDIA Hopper architecture in-depth*. NVIDIA Technical Blog. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., . . . Chintala, S. (2024). PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (pp. 929–947). ACM. https://doi.org/10.1145/3620665.3640366

Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., . . . Ballas, N. (2025). *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning* (arXiv:2506.09985). arXiv. https://doi.org/10.48550/arXiv.2506.09985

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., & Ballas, N. (2023). Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 15619–15629). IEEE. https://doi.org/10.1109/CVPR52729.2023.01499

Balestriero, R., & LeCun, Y. (2025). *LeJEPA: Provable and scalable self-supervised learning without the heuristics* (arXiv:2511.08544). arXiv. https://doi.org/10.48550/arXiv.2511.08544

Bar, A., Zhou, G., Tran, D., Darrell, T., & LeCun, Y. (2025). Navigation world models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 15791–15801). IEEE. https://doi.org/10.1109/CVPR52734.2025.01472

Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., & Ballas, N. (2024). Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=QaCCuDfBk2

Bharadhwaj, H., Xie, K., & Shkurti, F. (2020). Model-predictive control via cross-entropy and gradient-based optimization. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control* (Vol. 120, pp. 277–286). PMLR. https://proceedings.mlr.press/v120/bharadhwaj20a.html

Bhardwaj, M., Sundaralingam, B., Mousavian, A., Ratliff, N. D., Fox, D., Ramos, F., & Boots, B. (2022). STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation. In *Proceedings of the 5th Conference on Robot*

Learning (Vol. 164, pp. 750–759). PMLR. https://proceedings.mlr.press/v164/bhardwaj22a.html

Cheng, X., Zhang, Z., Zhou, Y., Ji, J., Jiang, J., Zhao, Z., Xiao, Z., Ye, Z., Huang, Y., Lai, R., Jin, H., Hou, B., Wu, M., Dong, Y., Yip, A., Wang, S., Yang, W., Miao, X., Chen, T., & Jia, Z. (2026). MPK: A compiler and runtime for mega-kernelizing tensor programs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)* (pp. 1909–1926). USENIX Association. https://www.usenix.org/conference/osdi26/presentation/cheng

Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., & Song, S. (2023). Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems XIX*. https://doi.org/10.15607/RSS.2023.XIX.026

Chua, K., Calandra, R., McAllister, R., & Levine, S. (2018). Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems 31* (pp. 4754–4765). Curran Associates. https://proceedings.neurips.cc/paper/2018/hash/3de568f8597b94bda53149c7d7f5958c-Abstract.html

Connor, R. J. (1987). Sample size for testing differences in proportions for the paired-sample design. *Biometrics, 43*(1), 207–211. https://doi.org/10.2307/2531961

Dao, T. (2024). FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=mZn2Xyh9Ec

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35* (pp. 16344–16359). Curran Associates. https://doi.org/10.52202/068431-1189

de Boer, P.-T., Kroese, D. P., Mannor, S., & Rubinstein, R. Y. (2005). A tutorial on the cross-entropy method. *Annals of Operations Research, 134*(1), 19–67. https://doi.org/10.1007/s10479-005-5724-z

Diamos, G., Sengupta, S., Catanzaro, B., Chrzanowski, M., Coates, A., Elsen, E., Engel, J., Hannun, A., & Satheesh, S. (2016). Persistent RNNs: Stashing recurrent weights on-chip. In *Proceedings of the 33rd International Conference on Machine Learning* (Vol. 48, pp. 2024–2033). PMLR. https://proceedings.mlr.press/v48/diamos16.html

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*. https://openreview.net/forum?id=YicbFdNTTy

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall. https://doi.org/10.1201/9780429246593

Florence, P., Lynch, C., Zeng, A., Ramirez, O. A., Wahid, A., Downs, L., Wong, A., Lee, J., Mordatch, I., & Tompson, J. (2022). Implicit behavioral cloning. In *Proceedings of the 5th Conference on Robot Learning* (Vol. 164, pp. 158–168). PMLR. https://proceedings.mlr.press/v164/florence22a.html

Gao, Y., & Xu, X. (2026). *Fast LeWorldModel* (arXiv:2606.26217). arXiv. https://doi.org/10.48550/arXiv.2606.26217

Gray, A. (2019, September 5). *Getting started with CUDA Graphs*. NVIDIA Technical Blog. https://developer.nvidia.com/blog/cuda-graphs/

Gupta, K., Stuart, J. A., & Owens, J. D. (2012). A study of persistent threads style GPU programming for GPGPU workloads. In *2012 Innovative Parallel Computing (InPar)* (pp. 1–14). IEEE. https://doi.org/10.1109/InPar.2012.6339596

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning* (Vol. 97, pp. 2555–2565). PMLR. https://proceedings.mlr.press/v97/hafner19a.html

Hansen, N. A., Su, H., & Wang, X. (2022). Temporal difference learning for model predictive control. In *Proceedings of the 39th International Conference on Machine Learning* (Vol. 162, pp. 8387–8406). PMLR. https://proceedings.mlr.press/v162/hansen22a.html

Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, robust world models for continuous control. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=Oxh5CstDJU

Hu, J., Zheng, Y., Bo, S., Wang, T., Dubost, F., Mottini, A., Liu, J., Srinivasan, A., Zhong, K., Qian, K., Gao, S., & Cui, Q. (2026). *Representation geometry matters for planning with JEPA world models* (arXiv:2608.16287). arXiv. https://doi.org/10.48550/arXiv.2608.16287

Huang, K., Lale, S., Rosolia, U., Shi, Y., & Anandkumar, A. (2021). *CEM-GD: Cross-entropy method with gradient descent planner for model-based reinforcement learning* (arXiv:2112.07746). arXiv. https://doi.org/10.48550/arXiv.2112.07746

Ivanov, A., Dryden, N., Ben-Nun, T., Li, S., & Hoefler, T. (2021). Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems 3* (MLSys 2021). https://doi.org/10.48550/arXiv.2007.00072

Jin, H., Hou, B., Wang, G., Lai, R., Chen, J., Ye, Z., Cai, Y., Dong, Y., Cheng, X., Zhang, Z., Zhao, Y., Huang, Y., Yang, L., Jiang, J., Oliaro, G., Ji, J., Miao, X., Grover, V., Mowry, T. C., . . . Chen, T. (2026). *Event Tensor: A unified abstraction for compiling dynamic megakernel* (arXiv:2604.13327). arXiv. https://doi.org/10.48550/arXiv.2604.13327

Kalamkar, D., Mudigere, D., Mellempudi, N., Das, D., Banerjee, K., Avancha, S., Vooturi, D. T., Jammalamadaka, N., Huang, J., Yuen, H., Yang, J., Park, J., Heinecke, A., Georganas, E., Srinivasan, S., Kundu, A., Smelyanskiy, M., Kaul, B., & Dubey, P. (2019). *A study of BFLOAT16 for deep learning training* (arXiv:1905.12322). arXiv. https://doi.org/10.48550/arXiv.1905.12322

Luo, W., Fan, R., Li, Z., Du, D., Wang, Q., & Chu, X. (2024). Benchmarking and dissecting the Nvidia Hopper GPU architecture. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (pp. 656–667). IEEE. https://doi.org/10.1109/IPDPS57955.2024.00064

Maes, L., Le Lidec, Q., Facury, L., Massaudi, N., Chaurasia, A., Capuano, F., Gao, R., Gillin, T., Haramati, D., Scieur, D., LeCun, Y., & Balestriero, R. (2026). *stable-worldmodel: A platform for reproducible world modeling research and evaluation* (arXiv:2605.21800). arXiv. https://doi.org/10.48550/arXiv.2605.21800

Maes, L., Le Lidec, Q., Scieur, D., LeCun, Y., & Balestriero, R. (2026). *LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels* (arXiv:2603.19312). arXiv. https://doi.org/10.48550/arXiv.2603.19312

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157. https://doi.org/10.1007/BF02295996

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., & Wu, H. (2018). Mixed precision training. In *International Conference on Learning Representations*. https://openreview.net/forum?id=r1gs9JgRZ

Nam, H., Le Lidec, Q., Maes, L., LeCun, Y., & Balestriero, R. (2026). *Causal-JEPA: Learning world models through object-level latent masking* (arXiv:2602.11389). arXiv. https://doi.org/10.48550/arXiv.2602.11389

Nguyen, H., Xu, X., & Huang, X. (2026). *Latent geometry beyond search: Amortizing planning in world models* (arXiv:2605.08732). arXiv. https://doi.org/10.48550/arXiv.2605.08732

Nrusimha, A., Brandon, W., Mishra, M., Shen, Y., Panda, R., Ragan-Kelley, J., & Kim, Y. (2025). *FlashFormer: Whole-model kernels for efficient low-batch inference* (arXiv:2505.22758). arXiv. https://doi.org/10.48550/arXiv.2505.22758

NVIDIA. (2022). *NVIDIA H100 Tensor Core GPU architecture* [White paper, V1.02].

NVIDIA. (2026). *CUDA programming guide* (Version 13.4.2). https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., . . . Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=a68SUt6zFt

Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 4172–4182). IEEE. https://doi.org/10.1109/ICCV51070.2023.00387

Pinneri, C., Sawant, S., Blaes, S., Achterhold, J., Stueckler, J., Rolinek, M., & Martius, G. (2021). Sample-efficient cross-entropy method for real-time planning. In *Proceedings of the 2020 Conference on Robot Learning* (Vol. 155, pp. 1049–1065). PMLR. https://proceedings.mlr.press/v155/pinneri21a.html

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., & Dean, J. (2023). Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems 5* (MLSys 2023). https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html

Rubinstein, R. (1999). The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability, 1*(2), 127–190. https://doi.org/10.1023/A:1010091220143

S V, J., Jalagam, S., LeCun, Y., & Sobal, V. (2023). *Gradient-based planning with world models* (arXiv:2312.17227). arXiv. https://doi.org/10.48550/arXiv.2312.17227

Salmon, J. K., Moraes, M. A., Dror, R. O., & Shaw, D. E. (2011). Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Article 16). ACM. https://doi.org/10.1145/2063384.2063405

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems 37* (pp. 68658–68685). Curran Associates. https://doi.org/10.52202/079017-2193

Sobal, U., Zhang, W., Cho, K., Balestriero, R., Rudner, T. G. J., & LeCun, Y. (2025). Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems 38* (pp. 49126–49162). Curran Associates. https://doi.org/10.52202/085713-1465

Spector, B. F., Arora, S., Singhal, A., Parthasarathy, A., Fu, D. Y., & Ré, C. (2025). ThunderKittens: Simple, fast, and *adorable* kernels. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=0fJfVOSUra

Spector, B., Juravsky, J., Sul, S., Dugan, O., Lim, D., Fu, D., Arora, S., & Ré, C. (2025, May 27). *Look ma, no bubbles! Designing a low-latency megakernel for Llama-1B*. Hazy Research Blog, Stanford University. https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles

Spector, B., Juravsky, J., Sul, S., Lim, D., Dugan, O., Arora, S., & Ré, C. (2025, September 28). *We bought the whole GPU, so we're damn well going to use the whole GPU*. Hazy Research Blog, Stanford University. https://hazyresearch.stanford.edu/blog/2025-09-28-tp-llama-main

Sundaralingam, B., Hari, S. K. S., Fishman, A., Garrett, C., Van Wyk, K., Blukis, V., Millane, A., Oleynikova, H., Handa, A., Ramos, F., Ratliff, N., & Fox, D. (2023). *cuRobo: Parallelized collision-free minimum-jerk robot motion generation* (arXiv:2310.17274). arXiv. https://doi.org/10.48550/arXiv.2310.17274

Terver, B., Yang, T.-Y., Ponce, J., Bardes, A., & LeCun, Y. (2026). What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2512.24497

Tillet, P., Kung, H. T., & Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (pp. 10–19). ACM. https://doi.org/10.1145/3315508.3329973

Vlahov, B., Gibson, J., Gandhi, M., & Theodorou, E. A. (2024). *MPPI-Generic: A CUDA library for stochastic trajectory optimization* (arXiv:2409.07563). arXiv. https://doi.org/10.48550/arXiv.2409.07563

Wang, T., & Ba, J. (2020). Exploring model-based planning with policy networks. In *International Conference on Learning Representations*. https://openreview.net/forum?id=H1exf64KwH

Wang, Y., Bounou, O., Zhou, G., Balestriero, R., Rudner, T. G. J., LeCun, Y., & Ren, M. (2026). *Temporal straightening for latent planning* (arXiv:2603.12231). arXiv. https://doi.org/10.48550/arXiv.2603.12231

Williams, G., Aldrich, A., & Theodorou, E. A. (2017). Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics, 40*(2), 344–357. https://doi.org/10.2514/1.G001921

Williams, G., Wagener, N., Goldfain, B., Drews, P., Rehg, J. M., Boots, B., & Theodorou, E. A. (2017). Information theoretic MPC for model-based reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 1714–1721). IEEE. https://doi.org/10.1109/ICRA.2017.7989202

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM, 52*(4), 65–76. https://doi.org/10.1145/1498765.1498785

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., & Jia, Z. (2025). Mirage: A multi-level superoptimizer for tensor programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)* (pp. 21–38). USENIX Association. https://www.usenix.org/conference/osdi25/presentation/wu-mengdi

Zhou, G., Pan, H., LeCun, Y., & Pinto, L. (2025). DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning* (Vol. 267, pp. 79115–79135). PMLR. https://proceedings.mlr.press/v267/zhou25t.html

Appendix A. Per-rung results

Table A1. Batch-1 end-to-end plan latency, final pass (ms). Plans from seeds 1001 to 1060 (seed 1000 dropped as warmup); step calls are the non-replanning calls of the same episodes.

rung	plans	p50	95% CI of p50	p90	mean	min	max	step calls	step p50
R0	73	622.4	605.5 – 626.7	673.8	622.7	555.5	758.6	1,569	2.14
R1	73	377.0	375.8 – 381.0	401.0	382.2	368.6	425.7	1,598	2.02
R2	72	410.2	406.2 – 416.3	433.3	413.5	385.6	492.5	1,569	2.11
R2b	72	182.1	177.3 – 185.7	204.3	177.2	112.4	259.1	1,575	1.51
R3	75	59.3	59.1 – 59.4	61.1	59.0	54.3	69.7	1,671	2.35
R3-CEM	75	82.1	81.7 – 82.5	83.6	82.1	78.2	86.1	1,642	1.59
R4	76	46.5	46.4 – 46.6	47.0	51.8	45.1	457.8	1,641	0.82

Table A2. Push-T successes per evaluation seed (100 episodes each) and the run that produced them.

rung	seed 0	seed 1	seed 2	total	run
R0	84	89	88	261	lewm_baseline_h100_e100_s0-1-2_20 261 016t191 013z
R1	86	89	88	263	ladder_r1_quality_h100_20 261 017t075 029z
R2	87	89	89	265	ladder_r2_quality_h100_20 261 017t075 029z
R2b	86	84	91	261	lewm_flash_h100_e100_s0-1-2_20 261 016t101 037z
R3	90	86	86	262	ladder_r3_quality_h100_20 261 017t092 213z
R3-CEM	83	86	92	261	ladder_r3cem_quality_h100_20 261 017t092 213z
R4	86	84	90	260	ladder_r4_quality_h100_20 261 017t092 213z

The per-seed totals vary by up to 9 episodes within a rung (R3-CEM: 83 to 92), more than any rung's total differs from the reference's. This is the seed-to-seed variation of the episode sets, and it is why the comparisons in Section 3.6 are paired by episode rather than made on totals.

Appendix B. Configuration

Table B1. Planner configuration. R0, R1 and R3-CEM use the CEM column; R2, R2b, R3 and R4 use the iCEM column.

setting	CEM	bucketed iCEM
candidates per iteration	300	max(⌈300 · 1.25^{-i}⌉, 60), rounded up to a multiple of 60: 300, 240, 240, 180, 180, 120, 120, 120, then 60 for 22 iterations
iterations	30	30
elites	top 30	top 30
retained elites	0	5 per iteration
model evaluations per plan	9,000	2,820
horizon / receding horizon / action block	5 / 5 / 5	5 / 5 / 5
initial variance scale	1.0	1.0
coloured noise, momentum, clipping, best-elite return	—	none (rejected on Push-T in the LeWM-Flash study)
plan output	mean of the final distribution	mean of the final distribution

Table B2. Execution settings per rung, as passed to the benchmark harness (modal/megakernel_app.py, LADDER).

rung	solver	matmul precision	rollout autocast	compile	other
R0	stable-worldmodel CEMSolver	highest (FP32)	none	no	as released
R1	CEMExactSolver	highest	none	no	cached embeddings, no host sync
R2	ICEMSolver	highest	none	no	—
R2b	ICEMSolver	high (TF32)	BF16	torch.compile, reduce-overhead, warmed at all five population sizes	—
R3	MegakernelICEMSolver	highest (encoder in FP32)	none (kernel numerics)	no	16-CTA clusters, 7 clusters, single-environment launch
R4	MegakernelICEMSolver, fused encoder	—	—	no	as R3; uint8 frames to the kernel, goal embedding cached by content hash
R3-CEM	MegakernelICEMSolver	highest	none	no	CEM schedule, no retained elites

Table B3. Megakernel configuration on the H100.

item	value
grid	7 clusters × 16 CTAs (batch 1) or 15 clusters × 8 CTAs (batched evaluation); one CTA per SM, all co-resident
CTA	256 threads: two warpgroups, each with its own two-slot weight ring; 255 registers per thread
shared memory	231,584 bytes per CTA (the H100 maximum)
weight panels	1,472 × 24 KB BF16 (192 × 64 or 64 × 192), pre-swizzled to the ThunderKittens shared-tile layout; 964 for the predictor, action encoder and projector, 508 for

item	value
	the ViT encoder and its projector
row tile	64 token rows per cluster chunk; two-tile data-parallel mode for batched launches
random numbers	Philox 4×32-10 (cuRAND), keyed by plan seed, iteration, environment, candidate and action element
limits	up to 300 candidates per environment and 100 environments per launch (buffer sizing)
primitives (measured)	cluster barrier 0.65 µs (16-CTA: 0.68 µs); grid barrier 1.6 µs; 64 × 64 epilogue 0.15 µs; empty launch 14 µs

Appendix C. Repeatability and the batched regime

Table C1. The ladder measured twice. Phase A ran on the R3 kernel build before the fused encoder existed; the final pass ran all seven rungs. Medians and p90 in ms.

rung	Phase A p50	Phase A p90	final p50	final p90	change in p50
R0	526.9	546.8	622.4	673.8	+18.1%
R1	391.4	433.5	377.0	401.0	−3.7%
R2	410.3	446.2	410.2	433.3	0.0%
R2b	196.3	236.0	182.1	204.3	−7.2%
R3	58.0	59.3	59.3	61.1	+2.2%
R3-CEM	81.7	83.4	82.1	83.6	+0.5%

Taking the Phase A medians instead of the final ones would change the headline ratios but not the conclusions: the reference would be 11.3 times slower than R4 rather than 13.4, caching would be worth 1.35 times rather than 1.65, and iCEM in eager PyTorch would still be slower than CEM.

The quality evaluations ran 100 environments per process, and the megakernel solves every replanning environment of a call in one launch. That regime is not the subject of this paper, but it is where the megakernel's throughput shows, and Table C2 records it for completeness.

Table C2. The batched evaluation regime (H100, 100-environment Push-T evaluation, 15 × 8-CTA clusters). LeWM-Flash solves replanning environments one after another; its rows for 26 and 100 environments are its single-environment solve times the count.

quantity	reference (CEM, FP32)	LeWM-Flash	megakernel (R3 build)
solve, 1 environment	about 590–670 ms	82.3 ms	44.1 ms
solve, 26 environments in one call	—	about 2.14 s	180 ms
solve, 100 environments in one call	—	about 8.2 s	651 ms
plan call 26 in the evaluation (21–27 environments)	12.5 – 17.1 s	2.51 – 3.02 s	0.28 – 0.34 s
plan call 1 in the evaluation (100 environments, includes warmup)	60.5 – 66.9 s	10.7 – 17.5 s	0.77 – 1.86 s
evaluation wall time per seed (100 episodes)	118 – 132 s	53.2 – 62.6 s	49.7 – 50.7 s

Once planning is fast, the batched evaluation's wall time is dominated by the 48 non-planning calls per episode, each about 105 ms for 100 environments in the policy's per-call CPU preprocessing, and by the simulator; the fused path of R4 removes most of the former.

Appendix D. Compute, cost and incident log

All GPU work ran on Modal, on single H100 80GB HBM3 containers for runs and on CPU containers for kernel builds. The study was approved with a budget of $10 for the megakernel alone, extended to $35 when the equal-hardware ladder was added; it used about $31.50 (Table D1).

Table D1. Approximate spend by stage.

stage	what ran	spend
Phase A batch-1 pass	R0–R3 and R3-CEM, 61 seeds each	about $9.50
final batch-1 pass	all seven rungs, 61 seeds each	about $11.60
everything else	kernel builds and development runs, parity checks, micro-benchmarks, the fused encoder, and all 300-episode quality runs	about $10.40 (by difference)
total		about $31.50

The batch-1 passes were the expensive part, and the reason is structural rather than computational: the harness starts a fresh process for each of the 61 seeds, which rebuilds the simulator, reloads the model and, for R2b, recompiles and warms the cost function at five shapes, so that most of a pass's GPU time is spent on setup rather than on the 75 plans it measures. A harness that keeps one process across seeds would cut the cost of a pass by an order of magnitude.

Five incidents cost time or money and changed the pipeline. The kernel extension did not rebuild after source changes, because the container mount did not preserve modification times; builds became content-addressed by the hash of the source, with the path of the built library recorded alongside. A printf on the failure path of the GEMM loop caused the compiler to serialise every WGMMA (warning C7510), and a run-time choice between mm and mma forms caused a related warning (C7515); both were removed. A torchvision build mismatched the pinned torch 2.11.0 and was re-pinned from the CUDA 12.8 index; an HDF5 compression plugin had to be imported explicitly before the dataset loaded; and one kernel build failed on a transient DNS error in the build container and was retried. None of these affected a reported result.

Appendix E. Code and data availability

All code, configuration, per-call latency records, per-episode outcomes, numerical parity records and analysis scripts are in the project repository (lewm-flash, branch antoinenguyen27/lewm-megakernel). The megakernel is megakernel/lewm_mk.cu with its packer (pack.py), runtime (runtime.py) and solver (solver.py); the parity, micro-benchmark and torch-baseline scripts are in the same directory. The ladder's rung definitions and the Modal entry points are in modal/megakernel_app.py; the benchmark harness is tools/benchmarks/lewm_plan_benchmark.py; the solvers of R1 and R2 are in lewm/solvers/. Per-call records are under results/runs/<run>/metrics.jsonl, the summaries under results/ladder/ and the parity and micro-benchmark records under results/megakernel/. The script docs/paper/src/compute_stats.py regenerates every latency and success statistic in this paper from those records, and docs/paper/src/make_figs.py every figure. The model is the released LeWM Push-T checkpoint, evaluated with stable-worldmodel's swm/PushT-v1 environment and its expert dataset; ThunderKittens is used at commit 67845f5f. Reproducing one rung's batch-1 latency requires one H100 and about $1.50 to $2 of GPU time with the current harness.